



Programs: Electrical & Computer Engineering

Course Number	COE838
Course Title	Systems on Chip Design
Semester/Year	Winter/2026
Instructor	Dr. Gul N. Khan

Lab Report No.	4
-----------------------	----------

Report Title	Interfacing Custom IP Cores to HPS/FPGA SOC Platform
---------------------	---

Section No.	COE838 - 052 (Section 5)
Group No.	N/A
Submission Date	March 11, 2026
Due Date	March 11, 2026 (4:00 PM)

Name	Student ID	Signature*
Syed Maisam Abbas	xxxx03255	

(Note: remove the first 4 digits from your student ID)

*By signing above you attest that you have contributed to this submission and confirm that all work you have contributed to this submission is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: <https://www.torontomu.ca/content/dam/senate/policies/pol60.pdf>

Table of Contents

Abstract.....	3
Introduction.....	3
Methodology.....	4
Results.....	6
Challenges.....	11
Conclusions.....	12
References.....	12
Appendix.....	13
soc_system.qsys.....	13
soc_system.h.....	15
makefile.....	54
hps_0.h.....	55
hps_0_arm_a9_0.h.....	56
hps_0_arm_a9_1.h.....	69
hps_0_bridges.h.....	81
main.c.....	93

ABSTRACT

Students can learn how to interface IP cores to HPS/FPGA systems in this lab. As a result, this lab requires the information from Lab 3. In order to connect these cores with HPS/FPGA-based SoCs, students will learn how to incorporate bespoke IP cores into their designs and build Avalon Memory Map (MM) slaves. After that, students will put this knowledge to use by integrating IPs into a SoC design of their choosing.

INTRODUCTION

The design flow for a DE1-SoC HPS/FPGA system is shown in Figure 1. The HPS/FPGA execution process is outlined by this figure through the various steps necessary to prototype a System-on-Chip (SoC) design using the DE1-SoC-based HPS/FPGA system. Turning the DE1-SoC board on (using the Quartus II Programmer) upload the.sof file to the FPGA. Use the Linux command to mount the USB after inserting it into the HPS, then copy the program to your home directory and run it. Your terminal output ought to look like Figure 1.

```
----- Iteration 27 -----
Reset done. Deasserting signal
Start successful
waiting for done
conversion done
0x0000001b * 0x0000001c = 0x000002f4. [Expected] 0x000002f4
[SUCCESSFUL]
-----
----- Iteration 28 -----
Reset done. Deasserting signal
Start successful
waiting for done
conversion done
0x0000001c * 0x0000001d = 0x0000032c. [Expected] 0x0000032c
[SUCCESSFUL]
-----
----- Iteration 29 -----
Reset done. Deasserting signal
Start successful
waiting for done
conversion done
0x0000001d * 0x0000001e = 0x00000366. [Expected] 0x00000366
[SUCCESSFUL]
-----
----- Iteration 30 -----
Reset done. Deasserting signal
Start successful
waiting for done
conversion done
0x0000001e * 0x0000001f = 0x000003a2. [Expected] 0x000003a2
[SUCCESSFUL]
-----
[TEST PASSED] 30/30
```

Figure 1: Sample Output from Multiplier SoC

METHODOLOGY

The objective of the previous lab experiment was to provide students with an introduction to the HPS/FPGA design process for system design implementation using the DE1-SoC. Each student will construct a hardware prototype in VHDL of the Cyclone V FPGA using both the Quartus II and Qsys design tools. Students will also gain experience programming in C for the embedded portion of an arrangement of software and hardware controlled by the Yocto Linux operating system. This laboratory will therefore teach students how to design prototypes for SSoCs from both the hardware and the software standpoint.

- **Software Design (Project Setup)**

Figure 2 shows a generic block diagram of the DE1-SoC Development Board for the LPM_MULT component. The DE1-SoC has a Cyclone V Device, which includes two different types of devices: a Field-Programmable Gate Array (FPGA) and a Hard Processor System (HPS). The HPS is an MPU (Vectored Interrupt Processor) with two ARM Cortex-A9 processors, on-chip memory, SDRAM, L3 interconnect, and all support and interface peripherals. This lab will use the HPS to run the software portion of your SoC design.

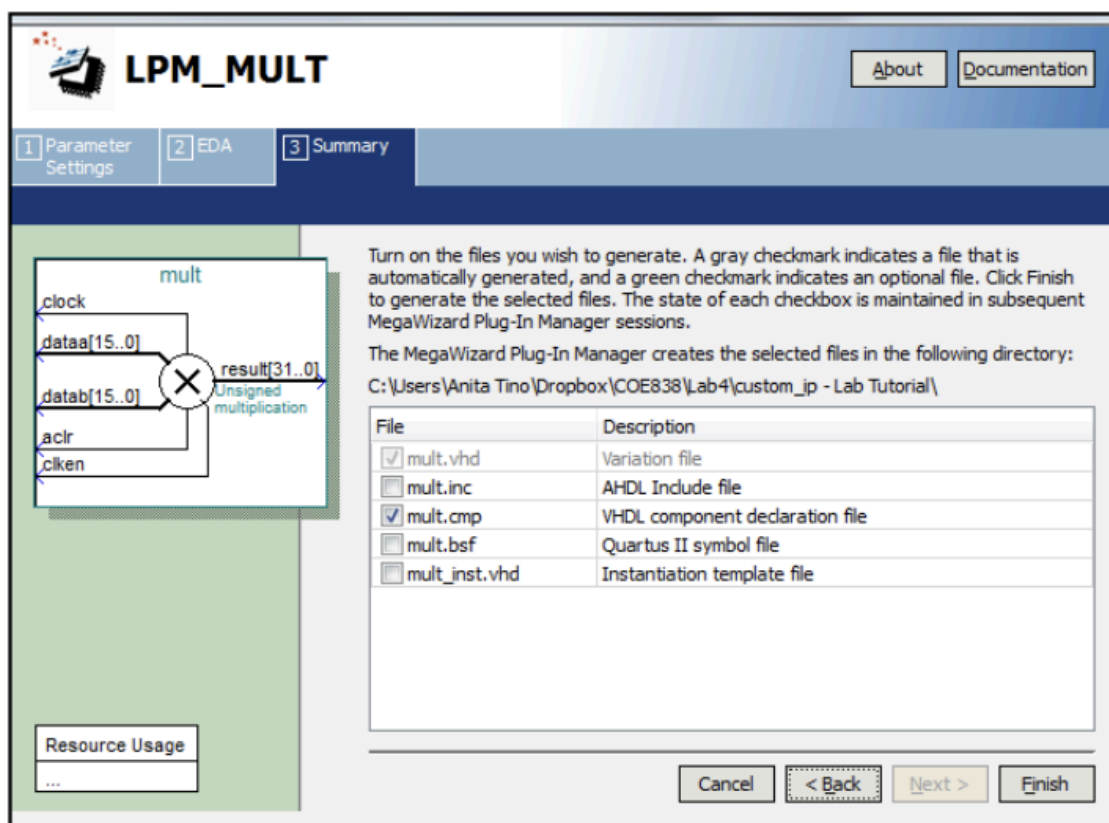


Figure 2: LPM_MULT Summary

- **Hardware Design** (Avalon MM Slave Interface Design)

Although the contents of the Avalon MM slave VHDL files appear to be typical VHDL files, they contain keywords that allow QSys to correctly identify Avalon bus signals. Specifically, the `avs_*` prefix tells QSys that this port is connected to the Avalon bus; other signals often indicate exporting conduits, clocks, resets, etc. Common signal names and attributes used by MM slaves for the Avalon bus are listed in Table I.

Name	Width	Direction	Comments
<code>avs_address</code>	≤ 64 bits	Input	Address of slave being accessed
<code>avs_read</code>	1 bit	Input	Read operation requested
<code>avs_write</code>	1 bit	Input	Write operation requested
<code>avs_readdata</code>	8, 16, 32 or 64 bits	Output	Data read from slave
<code>avs_writedata</code>	8, 16, 32 or 64 bits	Input	Data to be written to slave

Table I: Common Avalon Bus Signals

The `mult_data` bus is used to read and write data using the same idea. In order to facilitate easy access to the bus and conduits via the `alt_read_word()` and `alt_write_word()` APIs in the HPS while adhering to the Avalon bus constraints listed in Table I, all data-based ports are 32 bits. Figure 3 shows a visual depiction of the SoC and Avalon MM slaves covered here.

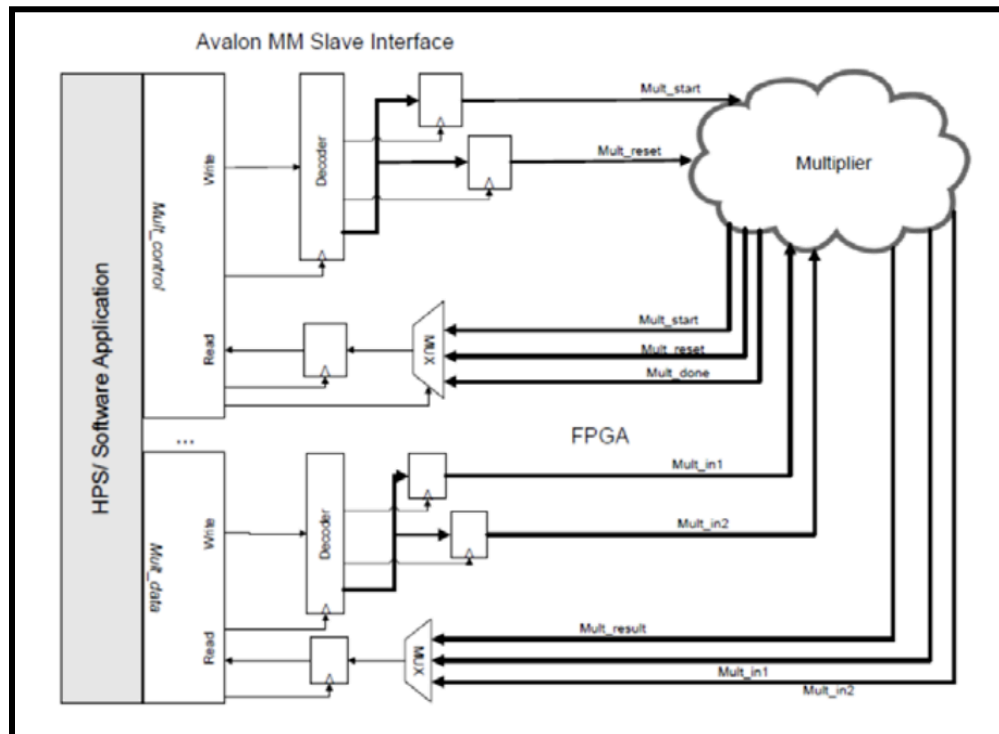


Figure 3: Multiplier SoC Avalon MM Slave Interface

- **HPS/FPGA Cyclone V Device (IP Wrapper, QSys Component Creation, SoC Design)**

The Cyclone V FPGA component includes all standard FPGA components (LUT, CLB, PLL, etc.) along with additional peripherals typically found on a standard development board for FPGAs; it also includes the shared memory controllers. For example, when you create a design using an FPGA, that FPGA can be used to prototype a hardware implementation for your final system-on-chip implementation. In Figure 3, the HPS and the blind environment both have dedicated sets of pins and peripheral interfaces in the device. Therefore, these pins do not have any kind of free sharing between either side of the application. To implement this prototyping activity, the FPGA will receive and send data to/from the HPS using AXI buses, bridges and Avalon master/slave devices.

RESULTS

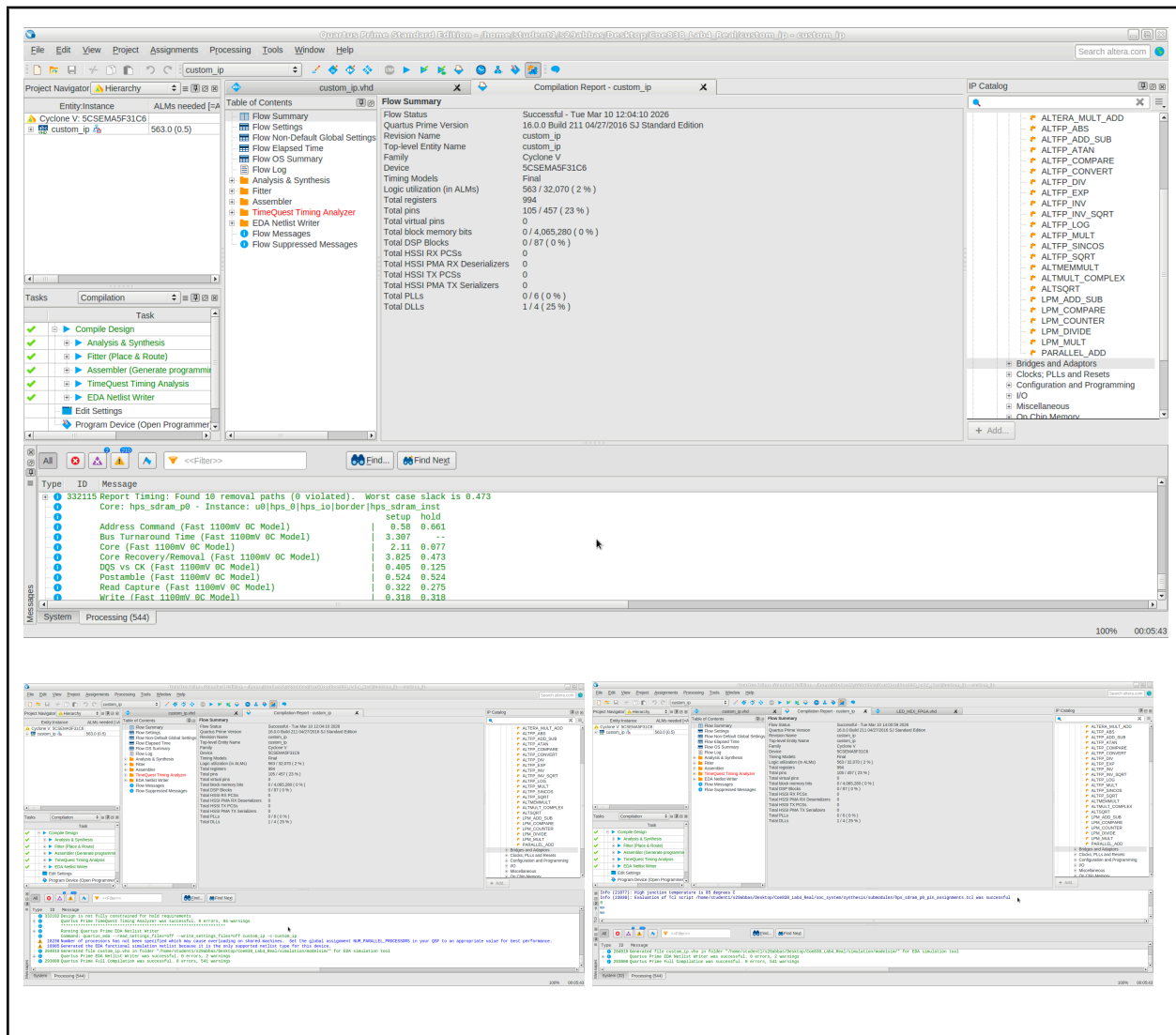


Figure 4: Compile Design (Analysis & Synthesis, Fitter, Assembler, Timing Analysis, EDA)


```

nios2_command_shell.sh
Version 16.0, Build 211
-----
bash-4.4$ ls
'COE_718.zip'  Downloads  Videos      mail          verilog.qsf
COE328        Lab1       coe718       minicom.log   verilog.qws
COE768        Music      db           output_files
COE838        Pictures   incremental_db simulation
Desktop       Public     lab5         ssh
Documents     Templates lab5.zip     verilog.qpf
bash-4.4$ cd Desktop/COE838_Lab4_Real
bash-4.4$ ls
c9_pin_model_dump.txt  div_control_hw.tcl      mult.cmp
custom_ip.qpf          div_data.vhd            mult.qip
custom_ip.qsf          div_data.vhd.bak        mult.vhd
custom_ip.vhd          div_data_hw.tcl         output_files
custom_ip.vhd.bak      div_unit.vhd            pin_assignment_DE1-SoC.tcl
db                     div_unit.vhd.bak        simulation
div.cmp               greybox_tmp             soc_system
div.qip               hps_isw_handoff         soc_system.qsys
div.vhd               hps_sdram_p0_all_pins.txt soc_system.sopcinfo
div_control.vhd        hps_sdram_p0_summary.csv software
div_control.vhd.bak    incremental_db

```

Figure 7: TCL Scripts using the NIOS II Command Shell (hps_sdram_p0_pin_assignments.tcl)

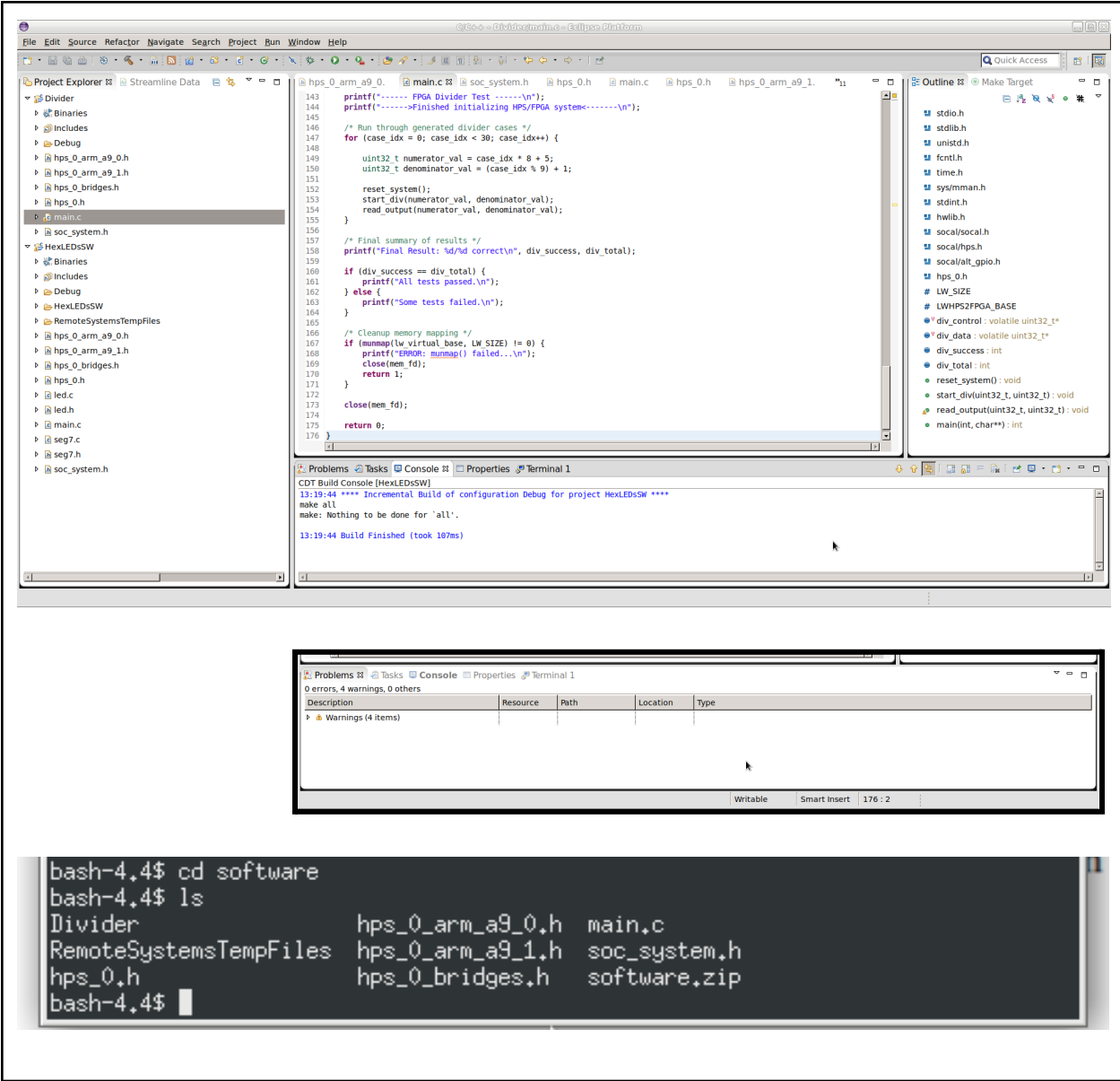


Figure 8: ARM Altera Eclipse for DS-5 compiled Build for the Altera SoC hardware API files

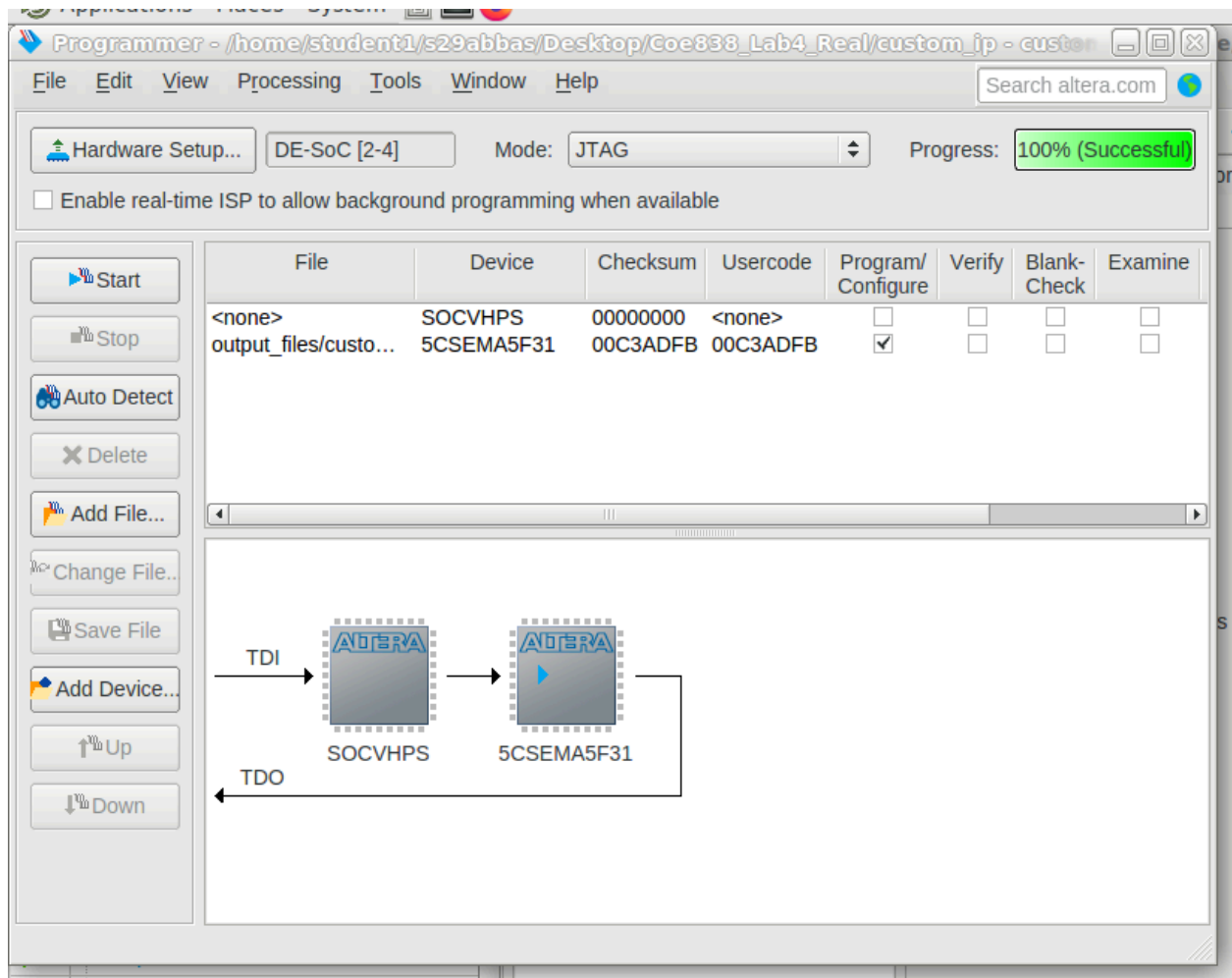


Figure 9: Programmer output configuration for the Hardware Setup (Programming the FPGA)

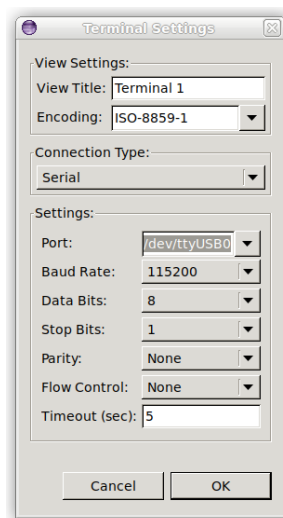


Figure 10: Port Configuration of the USB Drive for the DE1-SoC Board

CHALLENGES

Throughout the design and implementation of incorporating the FPGA fabric and HPS into a system, one significant complication was the hardware/software handoff due to Qsys being the source of hardware definitions used within software. Any changes to the FPGA devices (for example, changing the base address or adding additional peripheral devices) required a complete rebuilding of the system (or hardware/software) and related header files through the Nios II Command Shell. To properly synchronize the C-header file with its corresponding hardware mapping results in memory-mapped I/O where the ARM processor would write data into physical locations.

In order to ensure that the compiled binary could be correctly transferred using a USB drive to the /mnt directory, the commands had to be entered precisely using serial terminals (like the minicom) to manage the Yocto Linux environment. The C application must access the /dev/mem device to access the hardware registers, and any misaligned address offsets as defined in software will hang the system or cause bus errors. Lastly, the VHDL pin assignments of the Seven-Segment displays and LEDs had to correspond with the physical DE1-SoC hardware; matching the .tcl script, the compiled file may successfully compile, but the board may not have the expected physical output.

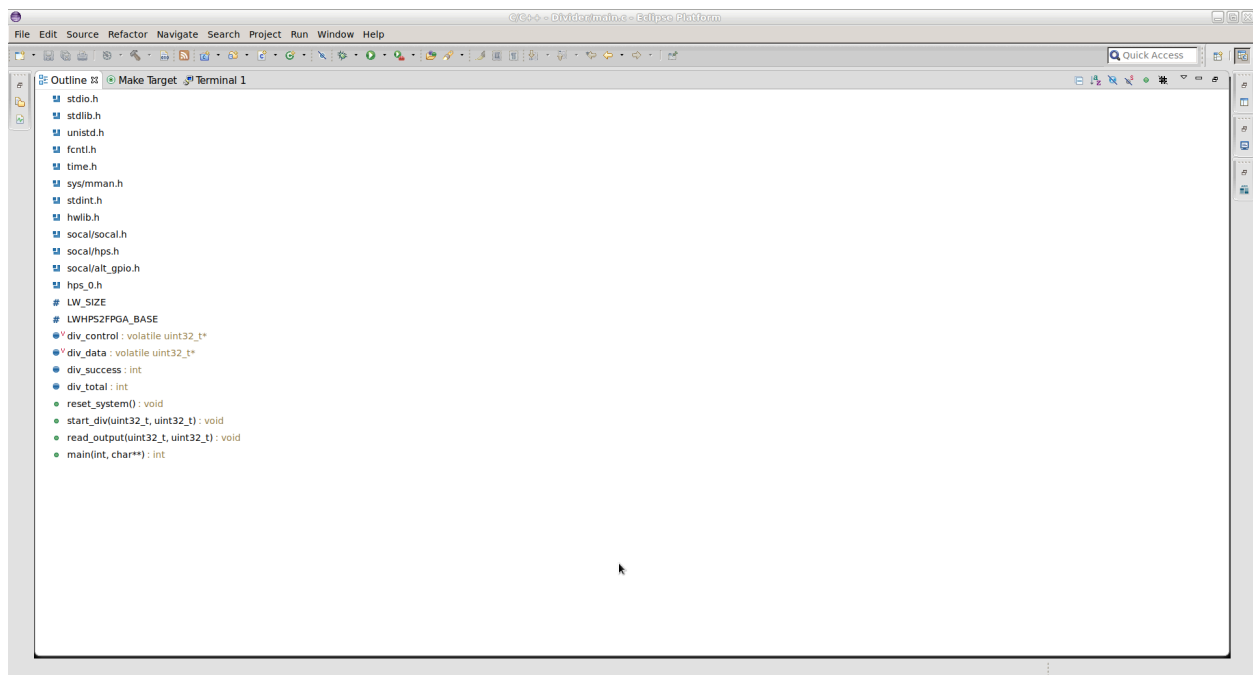


Figure 13: Project Workspace including the soc_system, HDL files, and Software.zip

CONCLUSIONS

The DE1-SoC was the hardware used to complete the design flow for an SoC, and Quartus II was used to describe the hardware, while Qsys was used to integrate the entire system design in order to establish a functional communication link between the ARM Cortex-A9 MPCore and the Cyclone V FPGA fabric. This exercise demonstrated the power of Memory-Mapped I/O. With memory-mapped I/O, software developed in the C programming language uses specific memory addresses to control hardware peripherals by performing reads and writes from those addresses. The successful completion of the HexLEDsSW program (as confirmed through use of the minicom terminal) provided evidence that the HPS is capable of driving the FPGA-side components, such as the Seven-Segment displays and LEDs, through the LWH2F bridge. By completing this laboratory exercise, practice has been gained in prototyping SSoCs, and it has shown that the design of modern embedded systems requires an interdisciplinary blend of VHDL hardware synthesis and Linux-based software development.

REFERENCES

Khan, Gul N. "COE838 / EE8221: Systems on Chip Design." *Systems-on-Chip Design*, Electrical, Computer and Biomedical Engineering, 5 January 2026, <http://www.ecb.torontomu.ca/~gnkhan>. Accessed 27 January 2026.

APPENDIX

soc_system.qsys

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

ENTITY custom_ip IS
    PORT( CLOCK_50, HPS_DDR3_RZQ, HPS_ENET_RX_CLK, HPS_ENET_RX_DV : IN STD_LOGIC;
          HPS_DDR3_ADDR : OUT STD_LOGIC_VECTOR(14 DOWNTO 0);
          HPS_DDR3_BA : OUT STD_LOGIC_VECTOR(2 DOWNTO 0);
          HPS_DDR3_CS_N : OUT STD_LOGIC;
          HPS_DDR3_CK_P, HPS_DDR3_CK_N, HPS_DDR3_CKE : OUT STD_LOGIC;
          HPS_USB_DIR, HPS_USB_NXT, HPS_USB_CLKOUT : IN STD_LOGIC;
          HPS_ENET_RX_DATA : IN STD_LOGIC_VECTOR(3 DOWNTO 0);
          HPS_SD_DATA, HPS_DDR3_DQS_N : INOUT STD_LOGIC_VECTOR(3 DOWNTO 0);
          HPS_DDR3_DQS_P : INOUT STD_LOGIC_VECTOR(3 DOWNTO 0);
          HPS_ENET_MDIO : INOUT STD_LOGIC;
          HPS_USB_DATA : INOUT STD_LOGIC_VECTOR(7 DOWNTO 0);
          HPS_DDR3_DQ : INOUT STD_LOGIC_VECTOR(31 DOWNTO 0);
          HPS_SD_CMD : INOUT STD_LOGIC;
          HPS_ENET_TX_DATA, HPS_DDR3_DM : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
          HPS_DDR3_ODT, HPS_DDR3_RAS_N, HPS_DDR3_RESET_N : OUT STD_LOGIC;
          HPS_DDR3_CAS_N, HPS_DDR3_WE_N : OUT STD_LOGIC;
          HPS_ENET_MDC, HPS_ENET_TX_EN : OUT STD_LOGIC;
          HPS_USB_STP, HPS_SD_CLK, HPS_ENET_GTX_CLK : OUT STD_LOGIC);
END ENTITY custom_ip;

ARCHITECTURE Behaviour of custom_ip IS
    component soc_system is
        port(clk_clk : in std_logic := 'X';
             hps_0_h2f_reset_reset_n : out std_logic;
             hps_io_hps_io_emacl_inst_TX_CLK : out std_logic;
             hps_io_hps_io_emacl_inst_TXD0 : out std_logic;
             hps_io_hps_io_emacl_inst_TXD1 : out std_logic;
             hps_io_hps_io_emacl_inst_TXD2 : out std_logic;
             hps_io_hps_io_emacl_inst_TXD3 : out std_logic;
             hps_io_hps_io_emacl_inst_RXD0 : in std_logic := 'X';
             hps_io_hps_io_emacl_inst_MDIO : inout std_logic := 'X';
             hps_io_hps_io_emacl_inst_MDC : out std_logic;
             hps_io_hps_io_emacl_inst_RX_CTL : in std_logic := 'X';
             hps_io_hps_io_emacl_inst_TX_CTL : out std_logic;
             hps_io_hps_io_emacl_inst_RX_CLK : in std_logic := 'X';
             hps_io_hps_io_emacl_inst_RXD1 : in std_logic := 'X';
             hps_io_hps_io_emacl_inst_RXD2 : in std_logic := 'X';
             hps_io_hps_io_emacl_inst_RXD3 : in std_logic := 'X';
             hps_io_hps_io_sdio_inst_CMD : inout std_logic := 'X';
             hps_io_hps_io_sdio_inst_D0 : inout std_logic := 'X';
             hps_io_hps_io_sdio_inst_D1 : inout std_logic := 'X';
             hps_io_hps_io_sdio_inst_CLK : out std_logic;
             hps_io_hps_io_sdio_inst_D2 : inout std_logic := 'X';
             hps_io_hps_io_sdio_inst_D3 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D0 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D1 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D2 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D3 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D4 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D5 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D6 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_D7 : inout std_logic := 'X';
             hps_io_hps_io_usb1_inst_CLK : in std_logic := 'X';
             hps_io_hps_io_usb1_inst_STP : out std_logic;
             hps_io_hps_io_usb1_inst_DIR : in std_logic := 'X';
             hps_io_hps_io_usb1_inst_NXT : in std_logic := 'X';
             memory_mem_a : out std_logic_vector(14 downto 0);
             memory_mem_ba : out std_logic_vector(2 downto 0);
             memory_mem_ck : out std_logic;
             memory_mem_ck_n : out std_logic;
             memory_mem_cke : out std_logic;
             memory_mem_cs_n : out std_logic;
             memory_mem_ras_n : out std_logic;
             memory_mem_cas_n : out std_logic;
             memory_mem_we_n : out std_logic;
             memory_mem_reset_n : out std_logic;
             memory_mem_dq : inout std_logic_vector(31 downto 0) := (others => 'X');
             memory_mem_dqs : inout std_logic_vector(3 downto 0) := (others => 'X');
             memory_mem_dqs_n : inout std_logic_vector(3 downto 0) := (others => 'X');
             memory_mem_odt : out std_logic;
             memory_mem_dm : out std_logic_vector(3 downto 0);
             memory_oct_rzqin : in std_logic := 'X');
```

```

        reset_reset_n          : in      std_logic              := 'X';
        mult_control_0_mult_control_m_start : out    std_logic_vector(31 downto 0);
        mult_control_0_mult_control_m_reset : out    std_logic_vector(31 downto 0);
        mult_control_0_mult_control_m_done  : in      std_logic_vector(31 downto 0) := (others => 'X');
        mult_data_0_mult_data_m_in1        : out    std_logic_vector(31 downto 0);
        mult_data_0_mult_data_m_in2        : out    std_logic_vector(31 downto 0);
        mult_data_0_mult_data_m_result     : in      std_logic_vector(31 downto 0) := (others => 'X')
    );
end COMPONENT soc_system;

COMPONENT mult_unit
    PORT( clk, reset, enable          : IN STD_LOGIC;
          mult_a, mult_b              : IN STD_LOGIC_VECTOR(15 DOWNTO 0);
          mult_done                   : OUT STD_LOGIC;
          mult_result                  : OUT STD_LOGIC_VECTOR(31 DOWNTO 0) := (others => '0'));
END COMPONENT;

SIGNAL reset_reset_n, done : STD_LOGIC ;
SIGNAL mult_input_reset : STD_LOGIC_VECTOR(31 DOWNTO 0);
SIGNAL mult_output_result, mult_input_start : STD_LOGIC_VECTOR(31 DOWNTO 0);
SIGNAL in1, in2 : STD_LOGIC_VECTOR(31 DOWNTO 0);

BEGIN
    u0 : component soc_system
        port map (
            clk_clk          => CLOCK_50,
            reset_reset_n    => '1',
            memory_mem_a     => HPS_DDR3_ADDR,
            memory_mem_ba    => HPS_DDR3_BA,
            memory_mem_ck     => HPS_DDR3_CK_P,
            memory_mem_ck_n  => HPS_DDR3_CK_N,
            memory_mem_cke    => HPS_DDR3_CKE,
            memory_mem_cs_n  => HPS_DDR3_CS_N,
            memory_mem_ras_n  => HPS_DDR3_RAS_N,
            memory_mem_cas_n  => HPS_DDR3_CAS_N,
            memory_mem_we_n   => HPS_DDR3_WE_N,
            memory_mem_reset_n => HPS_DDR3_RESET_N,
            memory_mem_dq     => HPS_DDR3_DQ,
            memory_mem_dqs    => HPS_DDR3_DQS_P,
            memory_mem_dqs_n  => HPS_DDR3_DQS_N,
            memory_mem_odt    => HPS_DDR3_ODT,
            memory_mem_dm     => HPS_DDR3_DM,
            memory_oct_rzqin  => HPS_DDR3_RZQ,
            hps_io_hps_io_emacl_inst_TX_CLK => HPS_ENET_GTX_CLK,
            hps_io_hps_io_emacl_inst_TXD0  => HPS_ENET_TX_DATA(0),
            hps_io_hps_io_emacl_inst_TXD1  => HPS_ENET_TX_DATA(1),
            hps_io_hps_io_emacl_inst_TXD2  => HPS_ENET_TX_DATA(2),
            hps_io_hps_io_emacl_inst_TXD3  => HPS_ENET_TX_DATA(3),
            hps_io_hps_io_emacl_inst_RXD0  => HPS_ENET_RX_DATA(0),
            hps_io_hps_io_emacl_inst_MDIO  => HPS_ENET_MDIO,
            hps_io_hps_io_emacl_inst_MDC   => HPS_ENET_MDC,
            hps_io_hps_io_emacl_inst_RX_CTL => HPS_ENET_RX_DV,
            hps_io_hps_io_emacl_inst_TX_CTL => HPS_ENET_TX_EN,
            hps_io_hps_io_emacl_inst_RX_CLK => HPS_ENET_RX_CLK,
            hps_io_hps_io_emacl_inst_RXD1  => HPS_ENET_RX_DATA(1),
            hps_io_hps_io_emacl_inst_RXD2  => HPS_ENET_RX_DATA(2),
            hps_io_hps_io_emacl_inst_RXD3  => HPS_ENET_RX_DATA(3),
            hps_io_hps_io_sdio_inst_CMD    => HPS_SD_CMD,
            hps_io_hps_io_sdio_inst_D0     => HPS_SD_DATA(0),
            hps_io_hps_io_sdio_inst_D1     => HPS_SD_DATA(1),
            hps_io_hps_io_sdio_inst_CLK    => HPS_SD_CLK,
            hps_io_hps_io_sdio_inst_D2     => HPS_SD_DATA(2),
            hps_io_hps_io_sdio_inst_D3     => HPS_SD_DATA(3),
            hps_io_hps_io_usb1_inst_D0     => HPS_USB_DATA(0),
            hps_io_hps_io_usb1_inst_D1     => HPS_USB_DATA(1),
            hps_io_hps_io_usb1_inst_D2     => HPS_USB_DATA(2),
            hps_io_hps_io_usb1_inst_D3     => HPS_USB_DATA(3),
            hps_io_hps_io_usb1_inst_D4     => HPS_USB_DATA(4),
            hps_io_hps_io_usb1_inst_D5     => HPS_USB_DATA(5),
            hps_io_hps_io_usb1_inst_D6     => HPS_USB_DATA(6),
            hps_io_hps_io_usb1_inst_D7     => HPS_USB_DATA(7),
            hps_io_hps_io_usb1_inst_CLK    => HPS_USB_CLKOUT,
            hps_io_hps_io_usb1_inst_STP    => HPS_USB_STP,
            hps_io_hps_io_usb1_inst_DIR    => HPS_USB_DIR,
            hps_io_hps_io_usb1_inst_NXT    => HPS_USB_NXT,
            hps_0_h2f_reset_reset_n       => reset_reset_n,
            mult_data_0_mult_data_m_result => mult_output_result,
            mult_control_0_mult_control_m_done => "00000000000000000000000000000000" & done,

```

```

        mult_data_0_mult_data_m_in1      => in1,
        mult_data_0_mult_data_m_in2      => in2,
        mult_control_0_mult_control_m_start => mult_input_start,
        mult_control_0_mult_control_m_reset => mult_input_reset
    );

    m0 : mult_unit
        PORT MAP( clk => CLOCK_50, reset => mult_input_reset(0), enable => mult_input_start(0),
            mult_a => in1(15 DOWNT0 0), mult_b => in2(15 DOWNT0 0),
            mult_done => done, mult_result => mult_output_result);

END Behaviour;

```

soc_system.h

```

#ifndef _ALTERA_SOC_SYSTEM_H_
#define _ALTERA_SOC_SYSTEM_H_

/*
 * This file was automatically generated by the swinfo2header utility.
 *
 * Created from SOPC Builder system 'soc_system' in
 * file './soc_system.sopcinfo'.
 */

/*
 * This file contains macros for all modules with masters in the system and
 * all the devices connected to those masters.
 * Macro names have unique prefixes to prevent duplicate names.
 */

/*
 * Macros for device 'div_data_0', class 'div_data'
 * Path to the device is from the master group 'hps_0'.
 * The macros are prefixed with 'HPS_0_DIV_DATA_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_DIV_DATA_0_COMPONENT_TYPE div_data
#define HPS_0_DIV_DATA_0_COMPONENT_NAME div_data_0
#define HPS_0_DIV_DATA_0_BASE 0x0
#define HPS_0_DIV_DATA_0_SPAN 64
#define HPS_0_DIV_DATA_0_END 0x3f

/*
 * Macros for device 'div_control_0', class 'div_control'
 * Path to the device is from the master group 'hps_0'.

```

```

* The macros are prefixed with 'HPS_0_DIV_CONTROL_0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_DIV_CONTROL_0_COMPONENT_TYPE div_control
#define HPS_0_DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define HPS_0_DIV_CONTROL_0_BASE 0x40
#define HPS_0_DIV_CONTROL_0_SPAN 64
#define HPS_0_DIV_CONTROL_0_END 0x7f

/*
* Macros for device 'hps_0_axi_sdran', class 'axi_sdran'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_AXI_SDRAM_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_COMPONENT_TYPE axi_sdran
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_COMPONENT_NAME hps_0_axi_sdran
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_BASE 0x0
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_SPAN 0x80000000
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_END 0x7fffffff
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_SIZE_MULTIPLE 1
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_SIZE_VALUE 1<<31
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_WRITABLE 1
#define
HPS_0_BRIDGES_HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_BRIDGES_HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_HEX 0
#define
HPS_0_BRIDGES_HPS_0_AXI_SDRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 31

/*
* Macros for device 'div_data_0', class 'div_data'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_DIV_DATA_0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_DIV_DATA_0_COMPONENT_TYPE div_data
#define HPS_0_BRIDGES_DIV_DATA_0_COMPONENT_NAME div_data_0
#define HPS_0_BRIDGES_DIV_DATA_0_BASE 0x0
#define HPS_0_BRIDGES_DIV_DATA_0_SPAN 64
#define HPS_0_BRIDGES_DIV_DATA_0_END 0x3f

```

```

/*
 * Macros for device 'div_control_0', class 'div_control'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_DIV_CONTROL_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_DIV_CONTROL_0_COMPONENT_TYPE div_control
#define HPS_0_BRIDGES_DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define HPS_0_BRIDGES_DIV_CONTROL_0_BASE 0x40
#define HPS_0_BRIDGES_DIV_CONTROL_0_SPAN 64
#define HPS_0_BRIDGES_DIV_CONTROL_0_END 0x7f

/*
 * Macros for device 'hps_0_gmac0', class 'stmmac'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_GMAC0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_GMAC0_COMPONENT_TYPE stmmac
#define HPS_0_BRIDGES_HPS_0_GMAC0_COMPONENT_NAME hps_0_gmac0
#define HPS_0_BRIDGES_HPS_0_GMAC0_BASE 0xff700000
#define HPS_0_BRIDGES_HPS_0_GMAC0_SPAN 8192
#define HPS_0_BRIDGES_HPS_0_GMAC0_END 0xff701fff

/*
 * Macros for device 'hps_0_gmac1', class 'stmmac'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_GMAC1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_GMAC1_COMPONENT_TYPE stmmac
#define HPS_0_BRIDGES_HPS_0_GMAC1_COMPONENT_NAME hps_0_gmac1
#define HPS_0_BRIDGES_HPS_0_GMAC1_BASE 0xff702000
#define HPS_0_BRIDGES_HPS_0_GMAC1_SPAN 8192
#define HPS_0_BRIDGES_HPS_0_GMAC1_END 0xff703fff

/*
 * Macros for device 'hps_0_sdmmc', class 'sdmmc'
 * Path to the device is from the master group 'hps_0_bridges'.

```

```

* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_SDMMC_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_SDMMC_COMPONENT_TYPE sdmmc
#define HPS_0_BRIDGES_HPS_0_SDMMC_COMPONENT_NAME hps_0_sdmmc
#define HPS_0_BRIDGES_HPS_0_SDMMC_BASE 0xff704000
#define HPS_0_BRIDGES_HPS_0_SDMMC_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_SDMMC_END 0xff704fff

/*
* Macros for device 'hps_0_qspi', class 'qspi'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_QSPI_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_QSPI_COMPONENT_TYPE qspi
#define HPS_0_BRIDGES_HPS_0_QSPI_COMPONENT_NAME hps_0_qspi
#define HPS_0_BRIDGES_HPS_0_QSPI_BASE 0xff705000
#define HPS_0_BRIDGES_HPS_0_QSPI_SPAN 256
#define HPS_0_BRIDGES_HPS_0_QSPI_END 0xff7050ff

/*
* Macros for device 'hps_0_fpgamgr', class 'fpgamgr'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_FPGAMGR_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_FPGAMGR_COMPONENT_TYPE fpgamgr
#define HPS_0_BRIDGES_HPS_0_FPGAMGR_COMPONENT_NAME hps_0_fpgamgr
#define HPS_0_BRIDGES_HPS_0_FPGAMGR_BASE 0xff706000
#define HPS_0_BRIDGES_HPS_0_FPGAMGR_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_FPGAMGR_END 0xff706fff

/*
* Macros for device 'hps_0_gpio0', class 'dw_gpio'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_GPIO0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_GPIO0_COMPONENT_TYPE dw_gpio

```

```

#define HPS_0_BRIDGES_HPS_0_GPIO0_COMPONENT_NAME hps_0_gpio0
#define HPS_0_BRIDGES_HPS_0_GPIO0_BASE 0xff708000
#define HPS_0_BRIDGES_HPS_0_GPIO0_SPAN 256
#define HPS_0_BRIDGES_HPS_0_GPIO0_END 0xff7080ff

/*
 * Macros for device 'hps_0_gpio1', class 'dw_gpio'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_GPIO1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_GPIO1_COMPONENT_TYPE dw_gpio
#define HPS_0_BRIDGES_HPS_0_GPIO1_COMPONENT_NAME hps_0_gpio1
#define HPS_0_BRIDGES_HPS_0_GPIO1_BASE 0xff709000
#define HPS_0_BRIDGES_HPS_0_GPIO1_SPAN 256
#define HPS_0_BRIDGES_HPS_0_GPIO1_END 0xff7090ff

/*
 * Macros for device 'hps_0_gpio2', class 'dw_gpio'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_GPIO2_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_GPIO2_COMPONENT_TYPE dw_gpio
#define HPS_0_BRIDGES_HPS_0_GPIO2_COMPONENT_NAME hps_0_gpio2
#define HPS_0_BRIDGES_HPS_0_GPIO2_BASE 0xff70a000
#define HPS_0_BRIDGES_HPS_0_GPIO2_SPAN 256
#define HPS_0_BRIDGES_HPS_0_GPIO2_END 0xff70a0ff

/*
 * Macros for device 'hps_0_l3regs', class 'l3regs'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_L3REGS_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_L3REGS_COMPONENT_TYPE l3regs
#define HPS_0_BRIDGES_HPS_0_L3REGS_COMPONENT_NAME hps_0_l3regs
#define HPS_0_BRIDGES_HPS_0_L3REGS_BASE 0xff800000
#define HPS_0_BRIDGES_HPS_0_L3REGS_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_L3REGS_END 0xff800fff

```

```

/*
 * Macros for device 'hps_0_nand0', class 'denali_nand'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_NAND0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_NAND0_COMPONENT_TYPE denali_nand
#define HPS_0_BRIDGES_HPS_0_NAND0_COMPONENT_NAME hps_0_nand0
#define HPS_0_BRIDGES_HPS_0_NAND0_BASE 0xff900000
#define HPS_0_BRIDGES_HPS_0_NAND0_SPAN 65536
#define HPS_0_BRIDGES_HPS_0_NAND0_END 0xff90ffff

/*
 * Macros for device 'hps_0_usb0', class 'usb'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_USB0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_USB0_COMPONENT_TYPE usb
#define HPS_0_BRIDGES_HPS_0_USB0_COMPONENT_NAME hps_0_usb0
#define HPS_0_BRIDGES_HPS_0_USB0_BASE 0xffb00000
#define HPS_0_BRIDGES_HPS_0_USB0_SPAN 262144
#define HPS_0_BRIDGES_HPS_0_USB0_END 0xffb3ffff

/*
 * Macros for device 'hps_0_usb1', class 'usb'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_USB1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_USB1_COMPONENT_TYPE usb
#define HPS_0_BRIDGES_HPS_0_USB1_COMPONENT_NAME hps_0_usb1
#define HPS_0_BRIDGES_HPS_0_USB1_BASE 0xffb40000
#define HPS_0_BRIDGES_HPS_0_USB1_SPAN 262144
#define HPS_0_BRIDGES_HPS_0_USB1_END 0xffb7ffff

/*
 * Macros for device 'hps_0_dcan0', class 'bosch_dcan'
 * Path to the device is from the master group 'hps_0_bridges'.

```

```

* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_DCAN0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_DCAN0_COMPONENT_TYPE bosch_dcan
#define HPS_0_BRIDGES_HPS_0_DCAN0_COMPONENT_NAME hps_0_dcan0
#define HPS_0_BRIDGES_HPS_0_DCAN0_BASE 0xffc00000
#define HPS_0_BRIDGES_HPS_0_DCAN0_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_DCAN0_END 0xffc00fff

/*
* Macros for device 'hps_0_dcan1', class 'bosch_dcan'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_DCAN1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_DCAN1_COMPONENT_TYPE bosch_dcan
#define HPS_0_BRIDGES_HPS_0_DCAN1_COMPONENT_NAME hps_0_dcan1
#define HPS_0_BRIDGES_HPS_0_DCAN1_BASE 0xffc01000
#define HPS_0_BRIDGES_HPS_0_DCAN1_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_DCAN1_END 0xffc01fff

/*
* Macros for device 'hps_0_uart0', class 'snps_uart'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_UART0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_UART0_COMPONENT_TYPE snps_uart
#define HPS_0_BRIDGES_HPS_0_UART0_COMPONENT_NAME hps_0_uart0
#define HPS_0_BRIDGES_HPS_0_UART0_BASE 0xffc02000
#define HPS_0_BRIDGES_HPS_0_UART0_SPAN 256
#define HPS_0_BRIDGES_HPS_0_UART0_END 0xffc020ff
#define HPS_0_BRIDGES_HPS_0_UART0_FIFO_DEPTH 128
#define HPS_0_BRIDGES_HPS_0_UART0_FIFO_HWFC 0
#define HPS_0_BRIDGES_HPS_0_UART0_FIFO_MODE 1
#define HPS_0_BRIDGES_HPS_0_UART0_FIFO_SWFC 0
#define HPS_0_BRIDGES_HPS_0_UART0_FREQ 100000000

/*
* Macros for device 'hps_0_uart1', class 'snps_uart'

```

```

* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_UART1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_UART1_COMPONENT_TYPE snps_uart
#define HPS_0_BRIDGES_HPS_0_UART1_COMPONENT_NAME hps_0_uart1
#define HPS_0_BRIDGES_HPS_0_UART1_BASE 0xffc03000
#define HPS_0_BRIDGES_HPS_0_UART1_SPAN 256
#define HPS_0_BRIDGES_HPS_0_UART1_END 0xffc030ff
#define HPS_0_BRIDGES_HPS_0_UART1_FIFO_DEPTH 128
#define HPS_0_BRIDGES_HPS_0_UART1_FIFO_HWFC 0
#define HPS_0_BRIDGES_HPS_0_UART1_FIFO_MODE 1
#define HPS_0_BRIDGES_HPS_0_UART1_FIFO_SWFC 0
#define HPS_0_BRIDGES_HPS_0_UART1_FREQ 100000000

/*
* Macros for device 'hps_0_i2c0', class 'designware_i2c'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_I2C0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_I2C0_COMPONENT_TYPE designware_i2c
#define HPS_0_BRIDGES_HPS_0_I2C0_COMPONENT_NAME hps_0_i2c0
#define HPS_0_BRIDGES_HPS_0_I2C0_BASE 0xffc04000
#define HPS_0_BRIDGES_HPS_0_I2C0_SPAN 256
#define HPS_0_BRIDGES_HPS_0_I2C0_END 0xffc040ff

/*
* Macros for device 'hps_0_i2c1', class 'designware_i2c'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_I2C1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_I2C1_COMPONENT_TYPE designware_i2c
#define HPS_0_BRIDGES_HPS_0_I2C1_COMPONENT_NAME hps_0_i2c1
#define HPS_0_BRIDGES_HPS_0_I2C1_BASE 0xffc05000
#define HPS_0_BRIDGES_HPS_0_I2C1_SPAN 256
#define HPS_0_BRIDGES_HPS_0_I2C1_END 0xffc050ff

/*

```

```

* Macros for device 'hps_0_i2c2', class 'designware_i2c'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_I2C2_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_I2C2_COMPONENT_TYPE designware_i2c
#define HPS_0_BRIDGES_HPS_0_I2C2_COMPONENT_NAME hps_0_i2c2
#define HPS_0_BRIDGES_HPS_0_I2C2_BASE 0xffc06000
#define HPS_0_BRIDGES_HPS_0_I2C2_SPAN 256
#define HPS_0_BRIDGES_HPS_0_I2C2_END 0xffc060ff

/*
* Macros for device 'hps_0_i2c3', class 'designware_i2c'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_I2C3_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_I2C3_COMPONENT_TYPE designware_i2c
#define HPS_0_BRIDGES_HPS_0_I2C3_COMPONENT_NAME hps_0_i2c3
#define HPS_0_BRIDGES_HPS_0_I2C3_BASE 0xffc07000
#define HPS_0_BRIDGES_HPS_0_I2C3_SPAN 256
#define HPS_0_BRIDGES_HPS_0_I2C3_END 0xffc070ff

/*
* Macros for device 'hps_0_timer0', class 'dw_apb_timer_sp'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_TIMER0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_TIMER0_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_BRIDGES_HPS_0_TIMER0_COMPONENT_NAME hps_0_timer0
#define HPS_0_BRIDGES_HPS_0_TIMER0_BASE 0xffc08000
#define HPS_0_BRIDGES_HPS_0_TIMER0_SPAN 256
#define HPS_0_BRIDGES_HPS_0_TIMER0_END 0xffc080ff

/*
* Macros for device 'hps_0_timer1', class 'dw_apb_timer_sp'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_TIMER1_'.
* The prefix is the master group descriptor and the slave descriptor.

```

```

*/
#define HPS_0_BRIDGES_HPS_0_TIMER1_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_BRIDGES_HPS_0_TIMER1_COMPONENT_NAME hps_0_timer1
#define HPS_0_BRIDGES_HPS_0_TIMER1_BASE 0xffc09000
#define HPS_0_BRIDGES_HPS_0_TIMER1_SPAN 256
#define HPS_0_BRIDGES_HPS_0_TIMER1_END 0xffc090ff

/*
* Macros for device 'hps_0_sdrcctl', class 'sdrcctl'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_SDRCTL_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_SDRCTL_COMPONENT_TYPE sdrcctl
#define HPS_0_BRIDGES_HPS_0_SDRCTL_COMPONENT_NAME hps_0_sdrcctl
#define HPS_0_BRIDGES_HPS_0_SDRCTL_BASE 0xffc25000
#define HPS_0_BRIDGES_HPS_0_SDRCTL_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_SDRCTL_END 0xffc25fff

/*
* Macros for device 'hps_0_timer2', class 'dw_apb_timer_osc'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_TIMER2_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_TIMER2_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_BRIDGES_HPS_0_TIMER2_COMPONENT_NAME hps_0_timer2
#define HPS_0_BRIDGES_HPS_0_TIMER2_BASE 0xffd00000
#define HPS_0_BRIDGES_HPS_0_TIMER2_SPAN 256
#define HPS_0_BRIDGES_HPS_0_TIMER2_END 0xffd000ff

/*
* Macros for device 'hps_0_timer3', class 'dw_apb_timer_osc'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_TIMER3_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_TIMER3_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_BRIDGES_HPS_0_TIMER3_COMPONENT_NAME hps_0_timer3
#define HPS_0_BRIDGES_HPS_0_TIMER3_BASE 0xffd01000

```

```

#define HPS_0_BRIDGES_HPS_0_TIMER3_SPAN 256
#define HPS_0_BRIDGES_HPS_0_TIMER3_END 0xffd010ff

/*
 * Macros for device 'hps_0_clkmgr', class 'clkmgr'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_CLKMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_CLKMGR_COMPONENT_TYPE clkmgr
#define HPS_0_BRIDGES_HPS_0_CLKMGR_COMPONENT_NAME hps_0_clkmgr
#define HPS_0_BRIDGES_HPS_0_CLKMGR_BASE 0xffd04000
#define HPS_0_BRIDGES_HPS_0_CLKMGR_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_CLKMGR_END 0xffd04fff

/*
 * Macros for device 'hps_0_rstmgr', class 'rstmgr'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_RSTMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_RSTMGR_COMPONENT_TYPE rstmgr
#define HPS_0_BRIDGES_HPS_0_RSTMGR_COMPONENT_NAME hps_0_rstmgr
#define HPS_0_BRIDGES_HPS_0_RSTMGR_BASE 0xffd05000
#define HPS_0_BRIDGES_HPS_0_RSTMGR_SPAN 256
#define HPS_0_BRIDGES_HPS_0_RSTMGR_END 0xffd050ff

/*
 * Macros for device 'hps_0_sysmgr', class 'sysmgr'
 * Path to the device is from the master group 'hps_0_bridges'.
 * The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_SYSMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_BRIDGES_HPS_0_SYSMGR_COMPONENT_TYPE sysmgr
#define HPS_0_BRIDGES_HPS_0_SYSMGR_COMPONENT_NAME hps_0_sysmgr
#define HPS_0_BRIDGES_HPS_0_SYSMGR_BASE 0xffd08000
#define HPS_0_BRIDGES_HPS_0_SYSMGR_SPAN 1024
#define HPS_0_BRIDGES_HPS_0_SYSMGR_END 0xffd083ff

/*

```

```

* Macros for device 'hps_0_dma', class 'dma'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_DMA_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_DMA_COMPONENT_TYPE dma
#define HPS_0_BRIDGES_HPS_0_DMA_COMPONENT_NAME hps_0_dma
#define HPS_0_BRIDGES_HPS_0_DMA_BASE 0xffe01000
#define HPS_0_BRIDGES_HPS_0_DMA_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_DMA_END 0xffe01fff

/*
* Macros for device 'hps_0_spim0', class 'spi'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_SPIM0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_SPIM0_COMPONENT_TYPE spi
#define HPS_0_BRIDGES_HPS_0_SPIM0_COMPONENT_NAME hps_0_spim0
#define HPS_0_BRIDGES_HPS_0_SPIM0_BASE 0xffff00000
#define HPS_0_BRIDGES_HPS_0_SPIM0_SPAN 256
#define HPS_0_BRIDGES_HPS_0_SPIM0_END 0xffff000ff

/*
* Macros for device 'hps_0_spim1', class 'spi'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_SPIM1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_SPIM1_COMPONENT_TYPE spi
#define HPS_0_BRIDGES_HPS_0_SPIM1_COMPONENT_NAME hps_0_spim1
#define HPS_0_BRIDGES_HPS_0_SPIM1_BASE 0xffff01000
#define HPS_0_BRIDGES_HPS_0_SPIM1_SPAN 256
#define HPS_0_BRIDGES_HPS_0_SPIM1_END 0xffff010ff

/*
* Macros for device 'hps_0_timer', class 'timer'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_TIMER_'.
* The prefix is the master group descriptor and the slave descriptor.

```

```

*/
#define HPS_0_BRIDGES_HPS_0_TIMER_COMPONENT_TYPE timer
#define HPS_0_BRIDGES_HPS_0_TIMER_COMPONENT_NAME hps_0_timer
#define HPS_0_BRIDGES_HPS_0_TIMER_BASE 0xffec600
#define HPS_0_BRIDGES_HPS_0_TIMER_SPAN 256
#define HPS_0_BRIDGES_HPS_0_TIMER_END 0xffec6ff

/*
* Macros for device 'hps_0_arm_gic_0', class 'arm_gic'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_ARM_GIC_0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_ARM_GIC_0_COMPONENT_TYPE arm_gic
#define HPS_0_BRIDGES_HPS_0_ARM_GIC_0_COMPONENT_NAME hps_0_arm_gic_0
#define HPS_0_BRIDGES_HPS_0_ARM_GIC_0_BASE 0xffed000
#define HPS_0_BRIDGES_HPS_0_ARM_GIC_0_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_ARM_GIC_0_END 0xffedfff

/*
* Macros for device 'hps_0_L2', class 'L2'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_L2_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_L2_COMPONENT_TYPE L2
#define HPS_0_BRIDGES_HPS_0_L2_COMPONENT_NAME hps_0_L2
#define HPS_0_BRIDGES_HPS_0_L2_BASE 0xffef000
#define HPS_0_BRIDGES_HPS_0_L2_SPAN 4096
#define HPS_0_BRIDGES_HPS_0_L2_END 0xffeffff

/*
* Macros for device 'hps_0_axi_ocram', class 'axi_ocram'
* Path to the device is from the master group 'hps_0_bridges'.
* The macros are prefixed with 'HPS_0_BRIDGES_HPS_0_AXI_OCRAM_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_COMPONENT_TYPE axi_ocram
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_COMPONENT_NAME hps_0_axi_ocram
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_BASE 0xfff0000

```

```

#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_SPAN 65536
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_END 0xffffffff
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_SIZE_MULTIPLE 1
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_SIZE_VALUE 1<<16
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_WRITABLE 1
#define
HPS_0_BRIDGES_HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_BRIDGES_HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_HEX 0
#define
HPS_0_BRIDGES_HPS_0_AXI_OCRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 16

/*
 * Macros for device 'hps_0_axi_sdram', class 'axi_sdram'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_COMPONENT_TYPE axi_sdram
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_COMPONENT_NAME hps_0_axi_sdram
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_BASE 0x0
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_SPAN 0x80000000
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_END 0x7fffffff
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_SIZE_MULTIPLE 1
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_SIZE_VALUE 1<<31
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_WRITABLE 1
#define
HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_HEX 0
#define
HPS_0_ARM_A9_0_HPS_0_AXI_SDRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH
31

/*
 * Macros for device 'div_data_0', class 'div_data'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_DIV_DATA_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_DIV_DATA_0_COMPONENT_TYPE div_data
#define HPS_0_ARM_A9_0_DIV_DATA_0_COMPONENT_NAME div_data_0

```

```

#define HPS_0_ARM_A9_0_DIV_DATA_0_BASE 0xff200000
#define HPS_0_ARM_A9_0_DIV_DATA_0_SPAN 64
#define HPS_0_ARM_A9_0_DIV_DATA_0_END 0xff20003f

/*
 * Macros for device 'div_control_0', class 'div_control'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_DIV_CONTROL_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_DIV_CONTROL_0_COMPONENT_TYPE div_control
#define HPS_0_ARM_A9_0_DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define HPS_0_ARM_A9_0_DIV_CONTROL_0_BASE 0xff200040
#define HPS_0_ARM_A9_0_DIV_CONTROL_0_SPAN 64
#define HPS_0_ARM_A9_0_DIV_CONTROL_0_END 0xff20007f

/*
 * Macros for device 'hps_0_gmac0', class 'stmmac'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_GMAC0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_GMAC0_COMPONENT_TYPE stmmac
#define HPS_0_ARM_A9_0_HPS_0_GMAC0_COMPONENT_NAME hps_0_gmac0
#define HPS_0_ARM_A9_0_HPS_0_GMAC0_BASE 0xff700000
#define HPS_0_ARM_A9_0_HPS_0_GMAC0_SPAN 8192
#define HPS_0_ARM_A9_0_HPS_0_GMAC0_END 0xff701fff

/*
 * Macros for device 'hps_0_gmac1', class 'stmmac'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_GMAC1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_GMAC1_COMPONENT_TYPE stmmac
#define HPS_0_ARM_A9_0_HPS_0_GMAC1_COMPONENT_NAME hps_0_gmac1
#define HPS_0_ARM_A9_0_HPS_0_GMAC1_BASE 0xff702000
#define HPS_0_ARM_A9_0_HPS_0_GMAC1_SPAN 8192
#define HPS_0_ARM_A9_0_HPS_0_GMAC1_END 0xff703fff

```

```

/*
 * Macros for device 'hps_0_sdmmc', class 'sdmmc'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_SDMMC_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_SDMMC_COMPONENT_TYPE sdmmc
#define HPS_0_ARM_A9_0_HPS_0_SDMMC_COMPONENT_NAME hps_0_sdmmc
#define HPS_0_ARM_A9_0_HPS_0_SDMMC_BASE 0xff704000
#define HPS_0_ARM_A9_0_HPS_0_SDMMC_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_SDMMC_END 0xff704fff

/*
 * Macros for device 'hps_0_qspi', class 'qspi'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_QSPI_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_QSPI_COMPONENT_TYPE qspi
#define HPS_0_ARM_A9_0_HPS_0_QSPI_COMPONENT_NAME hps_0_qspi
#define HPS_0_ARM_A9_0_HPS_0_QSPI_BASE 0xff705000
#define HPS_0_ARM_A9_0_HPS_0_QSPI_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_QSPI_END 0xff7050ff

/*
 * Macros for device 'hps_0_fpgamgr', class 'fpgamgr'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_FPGAMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_FPGAMGR_COMPONENT_TYPE fpgamgr
#define HPS_0_ARM_A9_0_HPS_0_FPGAMGR_COMPONENT_NAME hps_0_fpgamgr
#define HPS_0_ARM_A9_0_HPS_0_FPGAMGR_BASE 0xff706000
#define HPS_0_ARM_A9_0_HPS_0_FPGAMGR_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_FPGAMGR_END 0xff706fff

/*
 * Macros for device 'hps_0_gpio0', class 'dw_gpio'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_GPIO0_'.

```

```

* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_GPIO0_COMPONENT_TYPE dw_gpio
#define HPS_0_ARM_A9_0_HPS_0_GPIO0_COMPONENT_NAME hps_0_gpio0
#define HPS_0_ARM_A9_0_HPS_0_GPIO0_BASE 0xff708000
#define HPS_0_ARM_A9_0_HPS_0_GPIO0_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_GPIO0_END 0xff7080ff

/*
* Macros for device 'hps_0_gpio1', class 'dw_gpio'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_GPIO1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_GPIO1_COMPONENT_TYPE dw_gpio
#define HPS_0_ARM_A9_0_HPS_0_GPIO1_COMPONENT_NAME hps_0_gpio1
#define HPS_0_ARM_A9_0_HPS_0_GPIO1_BASE 0xff709000
#define HPS_0_ARM_A9_0_HPS_0_GPIO1_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_GPIO1_END 0xff7090ff

/*
* Macros for device 'hps_0_gpio2', class 'dw_gpio'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_GPIO2_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_GPIO2_COMPONENT_TYPE dw_gpio
#define HPS_0_ARM_A9_0_HPS_0_GPIO2_COMPONENT_NAME hps_0_gpio2
#define HPS_0_ARM_A9_0_HPS_0_GPIO2_BASE 0xff70a000
#define HPS_0_ARM_A9_0_HPS_0_GPIO2_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_GPIO2_END 0xff70a0ff

/*
* Macros for device 'hps_0_l3regs', class 'l3regs'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_L3REGS_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_L3REGS_COMPONENT_TYPE l3regs
#define HPS_0_ARM_A9_0_HPS_0_L3REGS_COMPONENT_NAME hps_0_l3regs

```

```

#define HPS_0_ARM_A9_0_HPS_0_L3REGS_BASE 0xff800000
#define HPS_0_ARM_A9_0_HPS_0_L3REGS_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_L3REGS_END 0xff800fff

/*
 * Macros for device 'hps_0_nand0', class 'denali_nand'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_NAND0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_NAND0_COMPONENT_TYPE denali_nand
#define HPS_0_ARM_A9_0_HPS_0_NAND0_COMPONENT_NAME hps_0_nand0
#define HPS_0_ARM_A9_0_HPS_0_NAND0_BASE 0xff900000
#define HPS_0_ARM_A9_0_HPS_0_NAND0_SPAN 65536
#define HPS_0_ARM_A9_0_HPS_0_NAND0_END 0xff90ffff

/*
 * Macros for device 'hps_0_usb0', class 'usb'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_USB0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_USB0_COMPONENT_TYPE usb
#define HPS_0_ARM_A9_0_HPS_0_USB0_COMPONENT_NAME hps_0_usb0
#define HPS_0_ARM_A9_0_HPS_0_USB0_BASE 0xffb00000
#define HPS_0_ARM_A9_0_HPS_0_USB0_SPAN 262144
#define HPS_0_ARM_A9_0_HPS_0_USB0_END 0xffb3ffff

/*
 * Macros for device 'hps_0_usb1', class 'usb'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_USB1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_USB1_COMPONENT_TYPE usb
#define HPS_0_ARM_A9_0_HPS_0_USB1_COMPONENT_NAME hps_0_usb1
#define HPS_0_ARM_A9_0_HPS_0_USB1_BASE 0xffb40000
#define HPS_0_ARM_A9_0_HPS_0_USB1_SPAN 262144
#define HPS_0_ARM_A9_0_HPS_0_USB1_END 0xffb7ffff

```

```

/*
 * Macros for device 'hps_0_dcan0', class 'bosch_dcan'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_DCAN0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_DCAN0_COMPONENT_TYPE bosch_dcan
#define HPS_0_ARM_A9_0_HPS_0_DCAN0_COMPONENT_NAME hps_0_dcan0
#define HPS_0_ARM_A9_0_HPS_0_DCAN0_BASE 0xffc00000
#define HPS_0_ARM_A9_0_HPS_0_DCAN0_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_DCAN0_END 0xffc00fff

/*
 * Macros for device 'hps_0_dcan1', class 'bosch_dcan'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_DCAN1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_DCAN1_COMPONENT_TYPE bosch_dcan
#define HPS_0_ARM_A9_0_HPS_0_DCAN1_COMPONENT_NAME hps_0_dcan1
#define HPS_0_ARM_A9_0_HPS_0_DCAN1_BASE 0xffc01000
#define HPS_0_ARM_A9_0_HPS_0_DCAN1_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_DCAN1_END 0xffc01fff

/*
 * Macros for device 'hps_0_uart0', class 'snps_uart'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_UART0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_UART0_COMPONENT_TYPE snps_uart
#define HPS_0_ARM_A9_0_HPS_0_UART0_COMPONENT_NAME hps_0_uart0
#define HPS_0_ARM_A9_0_HPS_0_UART0_BASE 0xffc02000
#define HPS_0_ARM_A9_0_HPS_0_UART0_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_UART0_END 0xffc020ff
#define HPS_0_ARM_A9_0_HPS_0_UART0_FIFO_DEPTH 128
#define HPS_0_ARM_A9_0_HPS_0_UART0_FIFO_HWFC 0
#define HPS_0_ARM_A9_0_HPS_0_UART0_FIFO_MODE 1
#define HPS_0_ARM_A9_0_HPS_0_UART0_FIFO_SWFC 0
#define HPS_0_ARM_A9_0_HPS_0_UART0_FREQ 100000000

```

```

/*
 * Macros for device 'hps_0_uart1', class 'snps_uart'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_UART1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_UART1_COMPONENT_TYPE snps_uart
#define HPS_0_ARM_A9_0_HPS_0_UART1_COMPONENT_NAME hps_0_uart1
#define HPS_0_ARM_A9_0_HPS_0_UART1_BASE 0xffc03000
#define HPS_0_ARM_A9_0_HPS_0_UART1_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_UART1_END 0xffc030ff
#define HPS_0_ARM_A9_0_HPS_0_UART1_FIFO_DEPTH 128
#define HPS_0_ARM_A9_0_HPS_0_UART1_FIFO_HWFC 0
#define HPS_0_ARM_A9_0_HPS_0_UART1_FIFO_MODE 1
#define HPS_0_ARM_A9_0_HPS_0_UART1_FIFO_SWFC 0
#define HPS_0_ARM_A9_0_HPS_0_UART1_FREQ 100000000

/*
 * Macros for device 'hps_0_i2c0', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_I2C0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_I2C0_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_0_HPS_0_I2C0_COMPONENT_NAME hps_0_i2c0
#define HPS_0_ARM_A9_0_HPS_0_I2C0_BASE 0xffc04000
#define HPS_0_ARM_A9_0_HPS_0_I2C0_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_I2C0_END 0xffc040ff

/*
 * Macros for device 'hps_0_i2c1', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_I2C1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_I2C1_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_0_HPS_0_I2C1_COMPONENT_NAME hps_0_i2c1
#define HPS_0_ARM_A9_0_HPS_0_I2C1_BASE 0xffc05000
#define HPS_0_ARM_A9_0_HPS_0_I2C1_SPAN 256

```

```

#define HPS_0_ARM_A9_0_HPS_0_I2C1_END 0xffc050ff

/*
 * Macros for device 'hps_0_i2c2', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_I2C2_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_I2C2_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_0_HPS_0_I2C2_COMPONENT_NAME hps_0_i2c2
#define HPS_0_ARM_A9_0_HPS_0_I2C2_BASE 0xffc06000
#define HPS_0_ARM_A9_0_HPS_0_I2C2_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_I2C2_END 0xffc060ff

/*
 * Macros for device 'hps_0_i2c3', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_I2C3_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_I2C3_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_0_HPS_0_I2C3_COMPONENT_NAME hps_0_i2c3
#define HPS_0_ARM_A9_0_HPS_0_I2C3_BASE 0xffc07000
#define HPS_0_ARM_A9_0_HPS_0_I2C3_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_I2C3_END 0xffc070ff

/*
 * Macros for device 'hps_0_timer0', class 'dw_apb_timer_sp'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_TIMER0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_TIMER0_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_ARM_A9_0_HPS_0_TIMER0_COMPONENT_NAME hps_0_timer0
#define HPS_0_ARM_A9_0_HPS_0_TIMER0_BASE 0xffc08000
#define HPS_0_ARM_A9_0_HPS_0_TIMER0_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_TIMER0_END 0xffc080ff

/*
 * Macros for device 'hps_0_timer1', class 'dw_apb_timer_sp'

```

```

* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_TIMER1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_TIMER1_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_ARM_A9_0_HPS_0_TIMER1_COMPONENT_NAME hps_0_timer1
#define HPS_0_ARM_A9_0_HPS_0_TIMER1_BASE 0xffc09000
#define HPS_0_ARM_A9_0_HPS_0_TIMER1_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_TIMER1_END 0xffc090ff

/*
* Macros for device 'hps_0_sdrcctl', class 'sdrcctl'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_SDRCTL_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_SDRCTL_COMPONENT_TYPE sdrcctl
#define HPS_0_ARM_A9_0_HPS_0_SDRCTL_COMPONENT_NAME hps_0_sdrcctl
#define HPS_0_ARM_A9_0_HPS_0_SDRCTL_BASE 0xffc25000
#define HPS_0_ARM_A9_0_HPS_0_SDRCTL_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_SDRCTL_END 0xffc25fff

/*
* Macros for device 'hps_0_timer2', class 'dw_apb_timer_osc'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_TIMER2_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_TIMER2_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_ARM_A9_0_HPS_0_TIMER2_COMPONENT_NAME hps_0_timer2
#define HPS_0_ARM_A9_0_HPS_0_TIMER2_BASE 0xffd00000
#define HPS_0_ARM_A9_0_HPS_0_TIMER2_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_TIMER2_END 0xffd000ff

/*
* Macros for device 'hps_0_timer3', class 'dw_apb_timer_osc'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_TIMER3_'.
* The prefix is the master group descriptor and the slave descriptor.
*/

```

```

#define HPS_0_ARM_A9_0_HPS_0_TIMER3_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_ARM_A9_0_HPS_0_TIMER3_COMPONENT_NAME hps_0_timer3
#define HPS_0_ARM_A9_0_HPS_0_TIMER3_BASE 0xffd01000
#define HPS_0_ARM_A9_0_HPS_0_TIMER3_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_TIMER3_END 0xffd010ff

/*
 * Macros for device 'hps_0_wd_timer0', class 'dw_wd_timer'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_WD_TIMER0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER0_COMPONENT_TYPE dw_wd_timer
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER0_COMPONENT_NAME hps_0_wd_timer0
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER0_BASE 0xffd02000
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER0_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER0_END 0xffd020ff

/*
 * Macros for device 'hps_0_wd_timer1', class 'dw_wd_timer'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_WD_TIMER1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER1_COMPONENT_TYPE dw_wd_timer
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER1_COMPONENT_NAME hps_0_wd_timer1
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER1_BASE 0xffd03000
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER1_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_WD_TIMER1_END 0xffd030ff

/*
 * Macros for device 'hps_0_clkmgr', class 'clkmgr'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_CLKMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_CLKMGR_COMPONENT_TYPE clkmgr
#define HPS_0_ARM_A9_0_HPS_0_CLKMGR_COMPONENT_NAME hps_0_clkmgr
#define HPS_0_ARM_A9_0_HPS_0_CLKMGR_BASE 0xffd04000
#define HPS_0_ARM_A9_0_HPS_0_CLKMGR_SPAN 4096

```

```

#define HPS_0_ARM_A9_0_HPS_0_CLKMGR_END 0xffd04fff

/*
 * Macros for device 'hps_0_rstmgr', class 'rstmgr'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_RSTMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_RSTMGR_COMPONENT_TYPE rstmgr
#define HPS_0_ARM_A9_0_HPS_0_RSTMGR_COMPONENT_NAME hps_0_rstmgr
#define HPS_0_ARM_A9_0_HPS_0_RSTMGR_BASE 0xffd05000
#define HPS_0_ARM_A9_0_HPS_0_RSTMGR_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_RSTMGR_END 0xffd050ff

/*
 * Macros for device 'hps_0_sysmgr', class 'sysmgr'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_SYSMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_SYSMGR_COMPONENT_TYPE sysmgr
#define HPS_0_ARM_A9_0_HPS_0_SYSMGR_COMPONENT_NAME hps_0_sysmgr
#define HPS_0_ARM_A9_0_HPS_0_SYSMGR_BASE 0xffd08000
#define HPS_0_ARM_A9_0_HPS_0_SYSMGR_SPAN 1024
#define HPS_0_ARM_A9_0_HPS_0_SYSMGR_END 0xffd083ff

/*
 * Macros for device 'hps_0_dma', class 'dma'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_DMA_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_DMA_COMPONENT_TYPE dma
#define HPS_0_ARM_A9_0_HPS_0_DMA_COMPONENT_NAME hps_0_dma
#define HPS_0_ARM_A9_0_HPS_0_DMA_BASE 0xffe01000
#define HPS_0_ARM_A9_0_HPS_0_DMA_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_DMA_END 0xffe01fff

/*
 * Macros for device 'hps_0_spim0', class 'spi'

```

```

* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_SPIM0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_SPIM0_COMPONENT_TYPE spi
#define HPS_0_ARM_A9_0_HPS_0_SPIM0_COMPONENT_NAME hps_0_spim0
#define HPS_0_ARM_A9_0_HPS_0_SPIM0_BASE 0xfff00000
#define HPS_0_ARM_A9_0_HPS_0_SPIM0_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_SPIM0_END 0xfff000ff

/*
* Macros for device 'hps_0_spim1', class 'spi'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_SPIM1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_SPIM1_COMPONENT_TYPE spi
#define HPS_0_ARM_A9_0_HPS_0_SPIM1_COMPONENT_NAME hps_0_spim1
#define HPS_0_ARM_A9_0_HPS_0_SPIM1_BASE 0xfff01000
#define HPS_0_ARM_A9_0_HPS_0_SPIM1_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_SPIM1_END 0xfff010ff

/*
* Macros for device 'hps_0_scu', class 'scu'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_SCU_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_0_HPS_0_SCU_COMPONENT_TYPE scu
#define HPS_0_ARM_A9_0_HPS_0_SCU_COMPONENT_NAME hps_0_scu
#define HPS_0_ARM_A9_0_HPS_0_SCU_BASE 0xfffec000
#define HPS_0_ARM_A9_0_HPS_0_SCU_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_SCU_END 0xfffec0ff

/*
* Macros for device 'hps_0_timer', class 'timer'
* Path to the device is from the master group 'hps_0_arm_a9_0'.
* The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_TIMER_'.
* The prefix is the master group descriptor and the slave descriptor.
*/

```

```

#define HPS_0_ARM_A9_0_HPS_0_TIMER_COMPONENT_TYPE timer
#define HPS_0_ARM_A9_0_HPS_0_TIMER_COMPONENT_NAME hps_0_timer
#define HPS_0_ARM_A9_0_HPS_0_TIMER_BASE 0xffec600
#define HPS_0_ARM_A9_0_HPS_0_TIMER_SPAN 256
#define HPS_0_ARM_A9_0_HPS_0_TIMER_END 0xffec6ff

/*
 * Macros for device 'hps_0_arm_gic_0', class 'arm_gic'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_ARM_GIC_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_ARM_GIC_0_COMPONENT_TYPE arm_gic
#define HPS_0_ARM_A9_0_HPS_0_ARM_GIC_0_COMPONENT_NAME hps_0_arm_gic_0
#define HPS_0_ARM_A9_0_HPS_0_ARM_GIC_0_BASE 0xffed000
#define HPS_0_ARM_A9_0_HPS_0_ARM_GIC_0_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_ARM_GIC_0_END 0xffedfff

/*
 * Macros for device 'hps_0_L2', class 'L2'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_L2_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_L2_COMPONENT_TYPE L2
#define HPS_0_ARM_A9_0_HPS_0_L2_COMPONENT_NAME hps_0_L2
#define HPS_0_ARM_A9_0_HPS_0_L2_BASE 0xffef000
#define HPS_0_ARM_A9_0_HPS_0_L2_SPAN 4096
#define HPS_0_ARM_A9_0_HPS_0_L2_END 0xffeffff

/*
 * Macros for device 'hps_0_axi_ocram', class 'axi_ocram'
 * Path to the device is from the master group 'hps_0_arm_a9_0'.
 * The macros are prefixed with 'HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_COMPONENT_TYPE axi_ocram
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_COMPONENT_NAME hps_0_axi_ocram
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_BASE 0xffff0000
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_SPAN 65536

```

```

#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_END 0xffffffff
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_SIZE_MULTIPLE 1
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_SIZE_VALUE 1<<16
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_WRITABLE 1
#define
HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_HEX 0
#define
HPS_0_ARM_A9_0_HPS_0_AXI_OCRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH
16

/*
 * Macros for device 'hps_0_axi_sdram', class 'axi_sdram'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_COMPONENT_TYPE axi_sdram
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_COMPONENT_NAME hps_0_axi_sdram
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_BASE 0x0
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_SPAN 0x80000000
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_END 0x7fffffff
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_SIZE_MULTIPLE 1
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_SIZE_VALUE 1<<31
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_WRITABLE 1
#define
HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_HEX 0
#define
HPS_0_ARM_A9_1_HPS_0_AXI_SDRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH
31

/*
 * Macros for device 'div_data_0', class 'div_data'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_DIV_DATA_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_DIV_DATA_0_COMPONENT_TYPE div_data
#define HPS_0_ARM_A9_1_DIV_DATA_0_COMPONENT_NAME div_data_0

```

```

#define HPS_0_ARM_A9_1_DIV_DATA_0_BASE 0xff200000
#define HPS_0_ARM_A9_1_DIV_DATA_0_SPAN 64
#define HPS_0_ARM_A9_1_DIV_DATA_0_END 0xff20003f

/*
 * Macros for device 'div_control_0', class 'div_control'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_DIV_CONTROL_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_DIV_CONTROL_0_COMPONENT_TYPE div_control
#define HPS_0_ARM_A9_1_DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define HPS_0_ARM_A9_1_DIV_CONTROL_0_BASE 0xff200040
#define HPS_0_ARM_A9_1_DIV_CONTROL_0_SPAN 64
#define HPS_0_ARM_A9_1_DIV_CONTROL_0_END 0xff20007f

/*
 * Macros for device 'hps_0_gmac0', class 'stmmac'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_GMAC0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_GMAC0_COMPONENT_TYPE stmmac
#define HPS_0_ARM_A9_1_HPS_0_GMAC0_COMPONENT_NAME hps_0_gmac0
#define HPS_0_ARM_A9_1_HPS_0_GMAC0_BASE 0xff700000
#define HPS_0_ARM_A9_1_HPS_0_GMAC0_SPAN 8192
#define HPS_0_ARM_A9_1_HPS_0_GMAC0_END 0xff701fff

/*
 * Macros for device 'hps_0_gmac1', class 'stmmac'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_GMAC1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_GMAC1_COMPONENT_TYPE stmmac
#define HPS_0_ARM_A9_1_HPS_0_GMAC1_COMPONENT_NAME hps_0_gmac1
#define HPS_0_ARM_A9_1_HPS_0_GMAC1_BASE 0xff702000
#define HPS_0_ARM_A9_1_HPS_0_GMAC1_SPAN 8192
#define HPS_0_ARM_A9_1_HPS_0_GMAC1_END 0xff703fff

```

```

/*
 * Macros for device 'hps_0_sdmmc', class 'sdmmc'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_SDMMC_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_SDMMC_COMPONENT_TYPE sdmmc
#define HPS_0_ARM_A9_1_HPS_0_SDMMC_COMPONENT_NAME hps_0_sdmmc
#define HPS_0_ARM_A9_1_HPS_0_SDMMC_BASE 0xff704000
#define HPS_0_ARM_A9_1_HPS_0_SDMMC_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_SDMMC_END 0xff704fff

/*
 * Macros for device 'hps_0_qspi', class 'qspi'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_QSPI_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_QSPI_COMPONENT_TYPE qspi
#define HPS_0_ARM_A9_1_HPS_0_QSPI_COMPONENT_NAME hps_0_qspi
#define HPS_0_ARM_A9_1_HPS_0_QSPI_BASE 0xff705000
#define HPS_0_ARM_A9_1_HPS_0_QSPI_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_QSPI_END 0xff7050ff

/*
 * Macros for device 'hps_0_fpgamgr', class 'fpgamgr'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_FPGAMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_FPGAMGR_COMPONENT_TYPE fpgamgr
#define HPS_0_ARM_A9_1_HPS_0_FPGAMGR_COMPONENT_NAME hps_0_fpgamgr
#define HPS_0_ARM_A9_1_HPS_0_FPGAMGR_BASE 0xff706000
#define HPS_0_ARM_A9_1_HPS_0_FPGAMGR_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_FPGAMGR_END 0xff706fff

/*
 * Macros for device 'hps_0_gpio0', class 'dw_gpio'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_GPIO0_'.

```

```

* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_GPIO0_COMPONENT_TYPE dw_gpio
#define HPS_0_ARM_A9_1_HPS_0_GPIO0_COMPONENT_NAME hps_0_gpio0
#define HPS_0_ARM_A9_1_HPS_0_GPIO0_BASE 0xff708000
#define HPS_0_ARM_A9_1_HPS_0_GPIO0_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_GPIO0_END 0xff7080ff

/*
* Macros for device 'hps_0_gpio1', class 'dw_gpio'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_GPIO1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_GPIO1_COMPONENT_TYPE dw_gpio
#define HPS_0_ARM_A9_1_HPS_0_GPIO1_COMPONENT_NAME hps_0_gpio1
#define HPS_0_ARM_A9_1_HPS_0_GPIO1_BASE 0xff709000
#define HPS_0_ARM_A9_1_HPS_0_GPIO1_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_GPIO1_END 0xff7090ff

/*
* Macros for device 'hps_0_gpio2', class 'dw_gpio'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_GPIO2_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_GPIO2_COMPONENT_TYPE dw_gpio
#define HPS_0_ARM_A9_1_HPS_0_GPIO2_COMPONENT_NAME hps_0_gpio2
#define HPS_0_ARM_A9_1_HPS_0_GPIO2_BASE 0xff70a000
#define HPS_0_ARM_A9_1_HPS_0_GPIO2_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_GPIO2_END 0xff70a0ff

/*
* Macros for device 'hps_0_l3regs', class 'l3regs'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_L3REGS_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_L3REGS_COMPONENT_TYPE l3regs
#define HPS_0_ARM_A9_1_HPS_0_L3REGS_COMPONENT_NAME hps_0_l3regs

```

```

#define HPS_0_ARM_A9_1_HPS_0_L3REGS_BASE 0xff800000
#define HPS_0_ARM_A9_1_HPS_0_L3REGS_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_L3REGS_END 0xff800fff

/*
 * Macros for device 'hps_0_nand0', class 'denali_nand'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_NAND0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_NAND0_COMPONENT_TYPE denali_nand
#define HPS_0_ARM_A9_1_HPS_0_NAND0_COMPONENT_NAME hps_0_nand0
#define HPS_0_ARM_A9_1_HPS_0_NAND0_BASE 0xff900000
#define HPS_0_ARM_A9_1_HPS_0_NAND0_SPAN 65536
#define HPS_0_ARM_A9_1_HPS_0_NAND0_END 0xff90ffff

/*
 * Macros for device 'hps_0_usb0', class 'usb'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_USB0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_USB0_COMPONENT_TYPE usb
#define HPS_0_ARM_A9_1_HPS_0_USB0_COMPONENT_NAME hps_0_usb0
#define HPS_0_ARM_A9_1_HPS_0_USB0_BASE 0xffb00000
#define HPS_0_ARM_A9_1_HPS_0_USB0_SPAN 262144
#define HPS_0_ARM_A9_1_HPS_0_USB0_END 0xffb3ffff

/*
 * Macros for device 'hps_0_usb1', class 'usb'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_USB1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_USB1_COMPONENT_TYPE usb
#define HPS_0_ARM_A9_1_HPS_0_USB1_COMPONENT_NAME hps_0_usb1
#define HPS_0_ARM_A9_1_HPS_0_USB1_BASE 0xffb40000
#define HPS_0_ARM_A9_1_HPS_0_USB1_SPAN 262144
#define HPS_0_ARM_A9_1_HPS_0_USB1_END 0xffb7ffff

```

```

/*
 * Macros for device 'hps_0_dcan0', class 'bosch_dcan'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_DCAN0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_DCAN0_COMPONENT_TYPE bosch_dcan
#define HPS_0_ARM_A9_1_HPS_0_DCAN0_COMPONENT_NAME hps_0_dcan0
#define HPS_0_ARM_A9_1_HPS_0_DCAN0_BASE 0xffc00000
#define HPS_0_ARM_A9_1_HPS_0_DCAN0_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_DCAN0_END 0xffc00fff

/*
 * Macros for device 'hps_0_dcan1', class 'bosch_dcan'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_DCAN1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_DCAN1_COMPONENT_TYPE bosch_dcan
#define HPS_0_ARM_A9_1_HPS_0_DCAN1_COMPONENT_NAME hps_0_dcan1
#define HPS_0_ARM_A9_1_HPS_0_DCAN1_BASE 0xffc01000
#define HPS_0_ARM_A9_1_HPS_0_DCAN1_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_DCAN1_END 0xffc01fff

/*
 * Macros for device 'hps_0_uart0', class 'snps_uart'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_UART0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_UART0_COMPONENT_TYPE snps_uart
#define HPS_0_ARM_A9_1_HPS_0_UART0_COMPONENT_NAME hps_0_uart0
#define HPS_0_ARM_A9_1_HPS_0_UART0_BASE 0xffc02000
#define HPS_0_ARM_A9_1_HPS_0_UART0_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_UART0_END 0xffc020ff
#define HPS_0_ARM_A9_1_HPS_0_UART0_FIFO_DEPTH 128
#define HPS_0_ARM_A9_1_HPS_0_UART0_FIFO_HWFC 0
#define HPS_0_ARM_A9_1_HPS_0_UART0_FIFO_MODE 1
#define HPS_0_ARM_A9_1_HPS_0_UART0_FIFO_SWFC 0
#define HPS_0_ARM_A9_1_HPS_0_UART0_FREQ 100000000

```

```

/*
 * Macros for device 'hps_0_uart1', class 'snps_uart'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_UART1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_UART1_COMPONENT_TYPE snps_uart
#define HPS_0_ARM_A9_1_HPS_0_UART1_COMPONENT_NAME hps_0_uart1
#define HPS_0_ARM_A9_1_HPS_0_UART1_BASE 0xffc03000
#define HPS_0_ARM_A9_1_HPS_0_UART1_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_UART1_END 0xffc030ff
#define HPS_0_ARM_A9_1_HPS_0_UART1_FIFO_DEPTH 128
#define HPS_0_ARM_A9_1_HPS_0_UART1_FIFO_HWFC 0
#define HPS_0_ARM_A9_1_HPS_0_UART1_FIFO_MODE 1
#define HPS_0_ARM_A9_1_HPS_0_UART1_FIFO_SWFC 0
#define HPS_0_ARM_A9_1_HPS_0_UART1_FREQ 100000000

/*
 * Macros for device 'hps_0_i2c0', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_I2C0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_I2C0_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_1_HPS_0_I2C0_COMPONENT_NAME hps_0_i2c0
#define HPS_0_ARM_A9_1_HPS_0_I2C0_BASE 0xffc04000
#define HPS_0_ARM_A9_1_HPS_0_I2C0_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_I2C0_END 0xffc040ff

/*
 * Macros for device 'hps_0_i2c1', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_I2C1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_I2C1_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_1_HPS_0_I2C1_COMPONENT_NAME hps_0_i2c1
#define HPS_0_ARM_A9_1_HPS_0_I2C1_BASE 0xffc05000
#define HPS_0_ARM_A9_1_HPS_0_I2C1_SPAN 256

```

```

#define HPS_0_ARM_A9_1_HPS_0_I2C1_END 0xffc050ff

/*
 * Macros for device 'hps_0_i2c2', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_I2C2_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_I2C2_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_1_HPS_0_I2C2_COMPONENT_NAME hps_0_i2c2
#define HPS_0_ARM_A9_1_HPS_0_I2C2_BASE 0xffc06000
#define HPS_0_ARM_A9_1_HPS_0_I2C2_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_I2C2_END 0xffc060ff

/*
 * Macros for device 'hps_0_i2c3', class 'designware_i2c'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_I2C3_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_I2C3_COMPONENT_TYPE designware_i2c
#define HPS_0_ARM_A9_1_HPS_0_I2C3_COMPONENT_NAME hps_0_i2c3
#define HPS_0_ARM_A9_1_HPS_0_I2C3_BASE 0xffc07000
#define HPS_0_ARM_A9_1_HPS_0_I2C3_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_I2C3_END 0xffc070ff

/*
 * Macros for device 'hps_0_timer0', class 'dw_apb_timer_sp'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_TIMER0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_TIMER0_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_ARM_A9_1_HPS_0_TIMER0_COMPONENT_NAME hps_0_timer0
#define HPS_0_ARM_A9_1_HPS_0_TIMER0_BASE 0xffc08000
#define HPS_0_ARM_A9_1_HPS_0_TIMER0_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_TIMER0_END 0xffc080ff

/*
 * Macros for device 'hps_0_timer1', class 'dw_apb_timer_sp'

```

```

* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_TIMER1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_TIMER1_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_ARM_A9_1_HPS_0_TIMER1_COMPONENT_NAME hps_0_timer1
#define HPS_0_ARM_A9_1_HPS_0_TIMER1_BASE 0xffc09000
#define HPS_0_ARM_A9_1_HPS_0_TIMER1_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_TIMER1_END 0xffc090ff

/*
* Macros for device 'hps_0_sdrcctl', class 'sdrcctl'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_SDRCTL_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_SDRCTL_COMPONENT_TYPE sdrcctl
#define HPS_0_ARM_A9_1_HPS_0_SDRCTL_COMPONENT_NAME hps_0_sdrcctl
#define HPS_0_ARM_A9_1_HPS_0_SDRCTL_BASE 0xffc25000
#define HPS_0_ARM_A9_1_HPS_0_SDRCTL_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_SDRCTL_END 0xffc25fff

/*
* Macros for device 'hps_0_timer2', class 'dw_apb_timer_osc'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_TIMER2_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_TIMER2_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_ARM_A9_1_HPS_0_TIMER2_COMPONENT_NAME hps_0_timer2
#define HPS_0_ARM_A9_1_HPS_0_TIMER2_BASE 0xffd00000
#define HPS_0_ARM_A9_1_HPS_0_TIMER2_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_TIMER2_END 0xffd000ff

/*
* Macros for device 'hps_0_timer3', class 'dw_apb_timer_osc'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_TIMER3_'.
* The prefix is the master group descriptor and the slave descriptor.
*/

```

```

#define HPS_0_ARM_A9_1_HPS_0_TIMER3_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_ARM_A9_1_HPS_0_TIMER3_COMPONENT_NAME hps_0_timer3
#define HPS_0_ARM_A9_1_HPS_0_TIMER3_BASE 0xffd01000
#define HPS_0_ARM_A9_1_HPS_0_TIMER3_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_TIMER3_END 0xffd010ff

/*
 * Macros for device 'hps_0_wd_timer0', class 'dw_wd_timer'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_WD_TIMER0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER0_COMPONENT_TYPE dw_wd_timer
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER0_COMPONENT_NAME hps_0_wd_timer0
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER0_BASE 0xffd02000
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER0_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER0_END 0xffd020ff

/*
 * Macros for device 'hps_0_wd_timer1', class 'dw_wd_timer'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_WD_TIMER1_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER1_COMPONENT_TYPE dw_wd_timer
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER1_COMPONENT_NAME hps_0_wd_timer1
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER1_BASE 0xffd03000
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER1_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_WD_TIMER1_END 0xffd030ff

/*
 * Macros for device 'hps_0_clkmgr', class 'clkmgr'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_CLKMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_CLKMGR_COMPONENT_TYPE clkmgr
#define HPS_0_ARM_A9_1_HPS_0_CLKMGR_COMPONENT_NAME hps_0_clkmgr
#define HPS_0_ARM_A9_1_HPS_0_CLKMGR_BASE 0xffd04000
#define HPS_0_ARM_A9_1_HPS_0_CLKMGR_SPAN 4096

```

```

#define HPS_0_ARM_A9_1_HPS_0_CLKMGR_END 0xffd04fff

/*
 * Macros for device 'hps_0_rstmgr', class 'rstmgr'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_RSTMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_RSTMGR_COMPONENT_TYPE rstmgr
#define HPS_0_ARM_A9_1_HPS_0_RSTMGR_COMPONENT_NAME hps_0_rstmgr
#define HPS_0_ARM_A9_1_HPS_0_RSTMGR_BASE 0xffd05000
#define HPS_0_ARM_A9_1_HPS_0_RSTMGR_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_RSTMGR_END 0xffd050ff

/*
 * Macros for device 'hps_0_sysmgr', class 'sysmgr'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_SYSMGR_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_SYSMGR_COMPONENT_TYPE sysmgr
#define HPS_0_ARM_A9_1_HPS_0_SYSMGR_COMPONENT_NAME hps_0_sysmgr
#define HPS_0_ARM_A9_1_HPS_0_SYSMGR_BASE 0xffd08000
#define HPS_0_ARM_A9_1_HPS_0_SYSMGR_SPAN 1024
#define HPS_0_ARM_A9_1_HPS_0_SYSMGR_END 0xffd083ff

/*
 * Macros for device 'hps_0_dma', class 'dma'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_DMA_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_DMA_COMPONENT_TYPE dma
#define HPS_0_ARM_A9_1_HPS_0_DMA_COMPONENT_NAME hps_0_dma
#define HPS_0_ARM_A9_1_HPS_0_DMA_BASE 0xffe01000
#define HPS_0_ARM_A9_1_HPS_0_DMA_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_DMA_END 0xffe01fff

/*
 * Macros for device 'hps_0_spim0', class 'spi'

```

```

* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_SPIM0_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_SPIM0_COMPONENT_TYPE spi
#define HPS_0_ARM_A9_1_HPS_0_SPIM0_COMPONENT_NAME hps_0_spim0
#define HPS_0_ARM_A9_1_HPS_0_SPIM0_BASE 0xfff00000
#define HPS_0_ARM_A9_1_HPS_0_SPIM0_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_SPIM0_END 0xfff000ff

/*
* Macros for device 'hps_0_spim1', class 'spi'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_SPIM1_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_SPIM1_COMPONENT_TYPE spi
#define HPS_0_ARM_A9_1_HPS_0_SPIM1_COMPONENT_NAME hps_0_spim1
#define HPS_0_ARM_A9_1_HPS_0_SPIM1_BASE 0xfff01000
#define HPS_0_ARM_A9_1_HPS_0_SPIM1_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_SPIM1_END 0xfff010ff

/*
* Macros for device 'hps_0_scu', class 'scu'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_SCU_'.
* The prefix is the master group descriptor and the slave descriptor.
*/
#define HPS_0_ARM_A9_1_HPS_0_SCU_COMPONENT_TYPE scu
#define HPS_0_ARM_A9_1_HPS_0_SCU_COMPONENT_NAME hps_0_scu
#define HPS_0_ARM_A9_1_HPS_0_SCU_BASE 0xfffec000
#define HPS_0_ARM_A9_1_HPS_0_SCU_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_SCU_END 0xfffec0ff

/*
* Macros for device 'hps_0_timer', class 'timer'
* Path to the device is from the master group 'hps_0_arm_a9_1'.
* The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_TIMER_'.
* The prefix is the master group descriptor and the slave descriptor.
*/

```

```

#define HPS_0_ARM_A9_1_HPS_0_TIMER_COMPONENT_TYPE timer
#define HPS_0_ARM_A9_1_HPS_0_TIMER_COMPONENT_NAME hps_0_timer
#define HPS_0_ARM_A9_1_HPS_0_TIMER_BASE 0xffec600
#define HPS_0_ARM_A9_1_HPS_0_TIMER_SPAN 256
#define HPS_0_ARM_A9_1_HPS_0_TIMER_END 0xffec6ff

/*
 * Macros for device 'hps_0_arm_gic_0', class 'arm_gic'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_ARM_GIC_0_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_ARM_GIC_0_COMPONENT_TYPE arm_gic
#define HPS_0_ARM_A9_1_HPS_0_ARM_GIC_0_COMPONENT_NAME hps_0_arm_gic_0
#define HPS_0_ARM_A9_1_HPS_0_ARM_GIC_0_BASE 0xffed000
#define HPS_0_ARM_A9_1_HPS_0_ARM_GIC_0_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_ARM_GIC_0_END 0xffedfff

/*
 * Macros for device 'hps_0_L2', class 'L2'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_L2_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_L2_COMPONENT_TYPE L2
#define HPS_0_ARM_A9_1_HPS_0_L2_COMPONENT_NAME hps_0_L2
#define HPS_0_ARM_A9_1_HPS_0_L2_BASE 0xffef000
#define HPS_0_ARM_A9_1_HPS_0_L2_SPAN 4096
#define HPS_0_ARM_A9_1_HPS_0_L2_END 0xffeffff

/*
 * Macros for device 'hps_0_axi_ocram', class 'axi_ocram'
 * Path to the device is from the master group 'hps_0_arm_a9_1'.
 * The macros are prefixed with 'HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_'.
 * The prefix is the master group descriptor and the slave descriptor.
 */
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_COMPONENT_TYPE axi_ocram
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_COMPONENT_NAME hps_0_axi_ocram
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_BASE 0xffff0000
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_SPAN 65536

```

```

#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_END 0xffffffff
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_SIZE_MULTIPLE 1
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_SIZE_VALUE 1<<16
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_WRITABLE 1
#define
HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_HEX 0
#define
HPS_0_ARM_A9_1_HPS_0_AXI_OCRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH
16

```

```

#endif /* _ALTERA_SOC_SYSTEM_H_ */

```

makefile

```

#####
##
# Automatically-generated file. Do not edit!
#####
##

```

```

-include ../makefile.init

```

```

RM := rm -rf

```

```

# All of the sources participating in the build are defined here

```

```

-include sources.mk
-include subdir.mk
-include objects.mk

```

```

ifneq ($(MAKECMDGOALS),clean)
ifneq ($(strip $(C_DEPS)),)
-include $(C_DEPS)
endif
endif

```

```

-include ../makefile.defs

```

```

# Add inputs and outputs from these tool invocations to the build variables

```

```
# All Target
all: Divider
```

```
# Tool invocations
```

```
Divider: $(OBJS) $(USER_OBJS)
    @echo 'Building target: $@'
    @echo 'Invoking: GCC C Linker'
    arm-linux-gnueabi-gcc -o "Divider" $(OBJS) $(USER_OBJS) $(LIBS)
    @echo 'Finished building target: $@'
    @echo ' '
```

```
# Other Targets
```

```
clean:
```

```
    -$(RM) $(OBJS) $(C_DEPS) $(EXECUTABLES) Divider
    -@echo ' '
```

```
.PHONY: all clean dependents
```

```
.SECONDARY:
```

```
-include ../makefile.targets
```

hps_0.h

```
#ifndef _ALTERA_HPS_0_H_
#define _ALTERA_HPS_0_H_
```

```
/*
```

```
 * This file was automatically generated by the swinfo2header utility.
```

```
 *
```

```
 * Created from SOPC Builder system 'soc_system' in
```

```
 * file './soc_system.sopcinfo'.
```

```
 */
```

```
/*
```

```
 * This file contains macros for module 'hps_0' and devices
```

```
 * connected to the following masters:
```

```
 * h2f_axi_master
```

```
 * h2f_lw_axi_master
```

```
 *
```

```
 * Do not include this header file and another header file created for a
```

```
 * different module or master group at the same time.
```

```

* Doing so may result in duplicate macro names.
* Instead, use the system header file which has macros with unique names.
*/

```

```

/*
* Macros for device 'div_data_0', class 'div_data'
* The macros are prefixed with 'DIV_DATA_0_'.
* The prefix is the slave descriptor.
*/
#define DIV_DATA_0_COMPONENT_TYPE div_data
#define DIV_DATA_0_COMPONENT_NAME div_data_0
#define DIV_DATA_0_BASE 0x0
#define DIV_DATA_0_SPAN 64
#define DIV_DATA_0_END 0x3f

```

```

/*
* Macros for device 'div_control_0', class 'div_control'
* The macros are prefixed with 'DIV_CONTROL_0_'.
* The prefix is the slave descriptor.
*/
#define DIV_CONTROL_0_COMPONENT_TYPE div_control
#define DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define DIV_CONTROL_0_BASE 0x40
#define DIV_CONTROL_0_SPAN 64
#define DIV_CONTROL_0_END 0x7f

```

```

#endif /* _ALTERA_HPS_0_H_ */

```

hps_0_arm_a9_0.h

```

#ifndef _ALTERA_HPS_0_ARM_A9_0_H_
#define _ALTERA_HPS_0_ARM_A9_0_H_

```

```

/*
* This file was automatically generated by the swinfo2header utility.
*
* Created from SOPC Builder system 'soc_system' in
* file './soc_system.sopcinfo'.
*/

```

```

/*
 * This file contains macros for module 'hps_0_arm_a9_0' and devices
 * connected to the following master:
 *   altera_axi_master
 *
 * Do not include this header file and another header file created for a
 * different module or master group at the same time.
 * Doing so may result in duplicate macro names.
 * Instead, use the system header file which has macros with unique names.
 */

/*
 * Macros for device 'hps_0_axi_sdrām', class 'axi_sdrām'
 * The macros are prefixed with 'HPS_0_AXI_SDRAM_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_AXI_SDRAM_COMPONENT_TYPE axi_sdrām
#define HPS_0_AXI_SDRAM_COMPONENT_NAME hps_0_axi_sdrām
#define HPS_0_AXI_SDRAM_BASE 0x0
#define HPS_0_AXI_SDRAM_SPAN 0x80000000
#define HPS_0_AXI_SDRAM_END 0x7ffffff
#define HPS_0_AXI_SDRAM_SIZE_MULTIPLE 1
#define HPS_0_AXI_SDRAM_SIZE_VALUE 1<<31
#define HPS_0_AXI_SDRAM_WRITABLE 1
#define HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_HEX 0
#define HPS_0_AXI_SDRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 31

/*
 * Macros for device 'div_data_0', class 'div_data'
 * The macros are prefixed with 'DIV_DATA_0_'.
 * The prefix is the slave descriptor.
 */
#define DIV_DATA_0_COMPONENT_TYPE div_data
#define DIV_DATA_0_COMPONENT_NAME div_data_0
#define DIV_DATA_0_BASE 0xff200000
#define DIV_DATA_0_SPAN 64
#define DIV_DATA_0_END 0xff20003f

/*

```

```

* Macros for device 'div_control_0', class 'div_control'
* The macros are prefixed with 'DIV_CONTROL_0_'.
* The prefix is the slave descriptor.
*/
#define DIV_CONTROL_0_COMPONENT_TYPE div_control
#define DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define DIV_CONTROL_0_BASE 0xff200040
#define DIV_CONTROL_0_SPAN 64
#define DIV_CONTROL_0_END 0xff20007f

/*
* Macros for device 'hps_0_gmac0', class 'stmmac'
* The macros are prefixed with 'HPS_0_GMAC0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_GMAC0_COMPONENT_TYPE stmmac
#define HPS_0_GMAC0_COMPONENT_NAME hps_0_gmac0
#define HPS_0_GMAC0_BASE 0xff700000
#define HPS_0_GMAC0_SPAN 8192
#define HPS_0_GMAC0_END 0xff701fff

/*
* Macros for device 'hps_0_gmac1', class 'stmmac'
* The macros are prefixed with 'HPS_0_GMAC1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_GMAC1_COMPONENT_TYPE stmmac
#define HPS_0_GMAC1_COMPONENT_NAME hps_0_gmac1
#define HPS_0_GMAC1_BASE 0xff702000
#define HPS_0_GMAC1_SPAN 8192
#define HPS_0_GMAC1_END 0xff703fff

/*
* Macros for device 'hps_0_sdmmc', class 'sdmmc'
* The macros are prefixed with 'HPS_0_SDMMC_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_SDMMC_COMPONENT_TYPE sdmmc
#define HPS_0_SDMMC_COMPONENT_NAME hps_0_sdmmc
#define HPS_0_SDMMC_BASE 0xff704000

```

```

#define HPS_0_SDMMC_SPAN 4096
#define HPS_0_SDMMC_END 0xff704fff

/*
 * Macros for device 'hps_0_qspi', class 'qspi'
 * The macros are prefixed with 'HPS_0_QSPI_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_QSPI_COMPONENT_TYPE qspi
#define HPS_0_QSPI_COMPONENT_NAME hps_0_qspi
#define HPS_0_QSPI_BASE 0xff705000
#define HPS_0_QSPI_SPAN 256
#define HPS_0_QSPI_END 0xff7050ff

/*
 * Macros for device 'hps_0_fpgamgr', class 'fpgamgr'
 * The macros are prefixed with 'HPS_0_FPGAMGR_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_FPGAMGR_COMPONENT_TYPE fpgamgr
#define HPS_0_FPGAMGR_COMPONENT_NAME hps_0_fpgamgr
#define HPS_0_FPGAMGR_BASE 0xff706000
#define HPS_0_FPGAMGR_SPAN 4096
#define HPS_0_FPGAMGR_END 0xff706fff

/*
 * Macros for device 'hps_0_gpio0', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO0_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO0_COMPONENT_NAME hps_0_gpio0
#define HPS_0_GPIO0_BASE 0xff708000
#define HPS_0_GPIO0_SPAN 256
#define HPS_0_GPIO0_END 0xff7080ff

/*
 * Macros for device 'hps_0_gpio1', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO1_'.
 * The prefix is the slave descriptor.

```

```

*/
#define HPS_0_GPIO1_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO1_COMPONENT_NAME hps_0_gpio1
#define HPS_0_GPIO1_BASE 0xff709000
#define HPS_0_GPIO1_SPAN 256
#define HPS_0_GPIO1_END 0xff7090ff

/*
 * Macros for device 'hps_0_gpio2', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO2_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO2_COMPONENT_NAME hps_0_gpio2
#define HPS_0_GPIO2_BASE 0xff70a000
#define HPS_0_GPIO2_SPAN 256
#define HPS_0_GPIO2_END 0xff70a0ff

/*
 * Macros for device 'hps_0_l3regs', class 'l3regs'
 * The macros are prefixed with 'HPS_0_L3REGS_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_L3REGS_COMPONENT_TYPE l3regs
#define HPS_0_L3REGS_COMPONENT_NAME hps_0_l3regs
#define HPS_0_L3REGS_BASE 0xff800000
#define HPS_0_L3REGS_SPAN 4096
#define HPS_0_L3REGS_END 0xff800fff

/*
 * Macros for device 'hps_0_nand0', class 'denali_nand'
 * The macros are prefixed with 'HPS_0_NAND0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_NAND0_COMPONENT_TYPE denali_nand
#define HPS_0_NAND0_COMPONENT_NAME hps_0_nand0
#define HPS_0_NAND0_BASE 0xff900000
#define HPS_0_NAND0_SPAN 65536
#define HPS_0_NAND0_END 0xff90ffff

```

```

/*
 * Macros for device 'hps_0_usb0', class 'usb'
 * The macros are prefixed with 'HPS_0_USB0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_USB0_COMPONENT_TYPE usb
#define HPS_0_USB0_COMPONENT_NAME hps_0_usb0
#define HPS_0_USB0_BASE 0xffb00000
#define HPS_0_USB0_SPAN 262144
#define HPS_0_USB0_END 0xffb3fff

/*
 * Macros for device 'hps_0_usb1', class 'usb'
 * The macros are prefixed with 'HPS_0_USB1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_USB1_COMPONENT_TYPE usb
#define HPS_0_USB1_COMPONENT_NAME hps_0_usb1
#define HPS_0_USB1_BASE 0xffb40000
#define HPS_0_USB1_SPAN 262144
#define HPS_0_USB1_END 0xffb7fff

/*
 * Macros for device 'hps_0_dcan0', class 'bosch_dcan'
 * The macros are prefixed with 'HPS_0_DCAN0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_DCAN0_COMPONENT_TYPE bosch_dcan
#define HPS_0_DCAN0_COMPONENT_NAME hps_0_dcan0
#define HPS_0_DCAN0_BASE 0xffc00000
#define HPS_0_DCAN0_SPAN 4096
#define HPS_0_DCAN0_END 0xffc00fff

/*
 * Macros for device 'hps_0_dcan1', class 'bosch_dcan'
 * The macros are prefixed with 'HPS_0_DCAN1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_DCAN1_COMPONENT_TYPE bosch_dcan
#define HPS_0_DCAN1_COMPONENT_NAME hps_0_dcan1

```

```

#define HPS_0_DCAN1_BASE 0xffc01000
#define HPS_0_DCAN1_SPAN 4096
#define HPS_0_DCAN1_END 0xffc01fff

/*
 * Macros for device 'hps_0_uart0', class 'snps_uart'
 * The macros are prefixed with 'HPS_0_UART0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_UART0_COMPONENT_TYPE snps_uart
#define HPS_0_UART0_COMPONENT_NAME hps_0_uart0
#define HPS_0_UART0_BASE 0xffc02000
#define HPS_0_UART0_SPAN 256
#define HPS_0_UART0_END 0xffc020ff
#define HPS_0_UART0_FIFO_DEPTH 128
#define HPS_0_UART0_FIFO_HWFC 0
#define HPS_0_UART0_FIFO_MODE 1
#define HPS_0_UART0_FIFO_SWFC 0
#define HPS_0_UART0_FREQ 100000000

/*
 * Macros for device 'hps_0_uart1', class 'snps_uart'
 * The macros are prefixed with 'HPS_0_UART1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_UART1_COMPONENT_TYPE snps_uart
#define HPS_0_UART1_COMPONENT_NAME hps_0_uart1
#define HPS_0_UART1_BASE 0xffc03000
#define HPS_0_UART1_SPAN 256
#define HPS_0_UART1_END 0xffc030ff
#define HPS_0_UART1_FIFO_DEPTH 128
#define HPS_0_UART1_FIFO_HWFC 0
#define HPS_0_UART1_FIFO_MODE 1
#define HPS_0_UART1_FIFO_SWFC 0
#define HPS_0_UART1_FREQ 100000000

/*
 * Macros for device 'hps_0_i2c0', class 'designware_i2c'
 * The macros are prefixed with 'HPS_0_I2C0_'.
 * The prefix is the slave descriptor.

```

```

*/
#define HPS_0_I2C0_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C0_COMPONENT_NAME hps_0_i2c0
#define HPS_0_I2C0_BASE 0xffc04000
#define HPS_0_I2C0_SPAN 256
#define HPS_0_I2C0_END 0xffc040ff

/*
* Macros for device 'hps_0_i2c1', class 'designware_i2c'
* The macros are prefixed with 'HPS_0_I2C1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_I2C1_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C1_COMPONENT_NAME hps_0_i2c1
#define HPS_0_I2C1_BASE 0xffc05000
#define HPS_0_I2C1_SPAN 256
#define HPS_0_I2C1_END 0xffc050ff

/*
* Macros for device 'hps_0_i2c2', class 'designware_i2c'
* The macros are prefixed with 'HPS_0_I2C2_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_I2C2_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C2_COMPONENT_NAME hps_0_i2c2
#define HPS_0_I2C2_BASE 0xffc06000
#define HPS_0_I2C2_SPAN 256
#define HPS_0_I2C2_END 0xffc060ff

/*
* Macros for device 'hps_0_i2c3', class 'designware_i2c'
* The macros are prefixed with 'HPS_0_I2C3_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_I2C3_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C3_COMPONENT_NAME hps_0_i2c3
#define HPS_0_I2C3_BASE 0xffc07000
#define HPS_0_I2C3_SPAN 256
#define HPS_0_I2C3_END 0xffc070ff

```

```

/*
 * Macros for device 'hps_0_timer0', class 'dw_apb_timer_sp'
 * The macros are prefixed with 'HPS_0_TIMER0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER0_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_TIMER0_COMPONENT_NAME hps_0_timer0
#define HPS_0_TIMER0_BASE 0xffc08000
#define HPS_0_TIMER0_SPAN 256
#define HPS_0_TIMER0_END 0xffc080ff

/*
 * Macros for device 'hps_0_timer1', class 'dw_apb_timer_sp'
 * The macros are prefixed with 'HPS_0_TIMER1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER1_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_TIMER1_COMPONENT_NAME hps_0_timer1
#define HPS_0_TIMER1_BASE 0xffc09000
#define HPS_0_TIMER1_SPAN 256
#define HPS_0_TIMER1_END 0xffc090ff

/*
 * Macros for device 'hps_0_sdrctl', class 'sdrctl'
 * The macros are prefixed with 'HPS_0_SDRCTL_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SDRCTL_COMPONENT_TYPE sdrctl
#define HPS_0_SDRCTL_COMPONENT_NAME hps_0_sdrctl
#define HPS_0_SDRCTL_BASE 0xffc25000
#define HPS_0_SDRCTL_SPAN 4096
#define HPS_0_SDRCTL_END 0xffc25fff

/*
 * Macros for device 'hps_0_timer2', class 'dw_apb_timer_osc'
 * The macros are prefixed with 'HPS_0_TIMER2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER2_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_TIMER2_COMPONENT_NAME hps_0_timer2

```

```

#define HPS_0_TIMER2_BASE 0xffd00000
#define HPS_0_TIMER2_SPAN 256
#define HPS_0_TIMER2_END 0xffd000ff

/*
 * Macros for device 'hps_0_timer3', class 'dw_apb_timer_osc'
 * The macros are prefixed with 'HPS_0_TIMER3_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER3_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_TIMER3_COMPONENT_NAME hps_0_timer3
#define HPS_0_TIMER3_BASE 0xffd01000
#define HPS_0_TIMER3_SPAN 256
#define HPS_0_TIMER3_END 0xffd010ff

/*
 * Macros for device 'hps_0_wd_timer0', class 'dw_wd_timer'
 * The macros are prefixed with 'HPS_0_WD_TIMER0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_WD_TIMER0_COMPONENT_TYPE dw_wd_timer
#define HPS_0_WD_TIMER0_COMPONENT_NAME hps_0_wd_timer0
#define HPS_0_WD_TIMER0_BASE 0xffd02000
#define HPS_0_WD_TIMER0_SPAN 256
#define HPS_0_WD_TIMER0_END 0xffd020ff

/*
 * Macros for device 'hps_0_wd_timer1', class 'dw_wd_timer'
 * The macros are prefixed with 'HPS_0_WD_TIMER1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_WD_TIMER1_COMPONENT_TYPE dw_wd_timer
#define HPS_0_WD_TIMER1_COMPONENT_NAME hps_0_wd_timer1
#define HPS_0_WD_TIMER1_BASE 0xffd03000
#define HPS_0_WD_TIMER1_SPAN 256
#define HPS_0_WD_TIMER1_END 0xffd030ff

/*
 * Macros for device 'hps_0_clkmgr', class 'clkmgr'
 * The macros are prefixed with 'HPS_0_CLKMGR_'.

```

```

* The prefix is the slave descriptor.
*/
#define HPS_0_CLKMGR_COMPONENT_TYPE clkmgr
#define HPS_0_CLKMGR_COMPONENT_NAME hps_0_clkmgr
#define HPS_0_CLKMGR_BASE 0xffd04000
#define HPS_0_CLKMGR_SPAN 4096
#define HPS_0_CLKMGR_END 0xffd04fff

/*
* Macros for device 'hps_0_rstmgr', class 'rstmgr'
* The macros are prefixed with 'HPS_0_RSTMGR_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_RSTMGR_COMPONENT_TYPE rstmgr
#define HPS_0_RSTMGR_COMPONENT_NAME hps_0_rstmgr
#define HPS_0_RSTMGR_BASE 0xffd05000
#define HPS_0_RSTMGR_SPAN 256
#define HPS_0_RSTMGR_END 0xffd050ff

/*
* Macros for device 'hps_0_sysmgr', class 'sysmgr'
* The macros are prefixed with 'HPS_0_SYSMGR_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_SYSMGR_COMPONENT_TYPE sysmgr
#define HPS_0_SYSMGR_COMPONENT_NAME hps_0_sysmgr
#define HPS_0_SYSMGR_BASE 0xffd08000
#define HPS_0_SYSMGR_SPAN 1024
#define HPS_0_SYSMGR_END 0xffd083ff

/*
* Macros for device 'hps_0_dma', class 'dma'
* The macros are prefixed with 'HPS_0_DMA_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_DMA_COMPONENT_TYPE dma
#define HPS_0_DMA_COMPONENT_NAME hps_0_dma
#define HPS_0_DMA_BASE 0xffe01000
#define HPS_0_DMA_SPAN 4096
#define HPS_0_DMA_END 0xffe01fff

```

```

/*
 * Macros for device 'hps_0_spim0', class 'spi'
 * The macros are prefixed with 'HPS_0_SPIM0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SPIM0_COMPONENT_TYPE spi
#define HPS_0_SPIM0_COMPONENT_NAME hps_0_spim0
#define HPS_0_SPIM0_BASE 0xfff00000
#define HPS_0_SPIM0_SPAN 256
#define HPS_0_SPIM0_END 0xfff000ff

/*
 * Macros for device 'hps_0_spim1', class 'spi'
 * The macros are prefixed with 'HPS_0_SPIM1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SPIM1_COMPONENT_TYPE spi
#define HPS_0_SPIM1_COMPONENT_NAME hps_0_spim1
#define HPS_0_SPIM1_BASE 0xfff01000
#define HPS_0_SPIM1_SPAN 256
#define HPS_0_SPIM1_END 0xfff010ff

/*
 * Macros for device 'hps_0_scu', class 'scu'
 * The macros are prefixed with 'HPS_0_SCU_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SCU_COMPONENT_TYPE scu
#define HPS_0_SCU_COMPONENT_NAME hps_0_scu
#define HPS_0_SCU_BASE 0xfffec000
#define HPS_0_SCU_SPAN 256
#define HPS_0_SCU_END 0xfffec0ff

/*
 * Macros for device 'hps_0_timer', class 'timer'
 * The macros are prefixed with 'HPS_0_TIMER_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER_COMPONENT_TYPE timer

```

```

#define HPS_0_TIMER_COMPONENT_NAME hps_0_timer
#define HPS_0_TIMER_BASE 0xffec600
#define HPS_0_TIMER_SPAN 256
#define HPS_0_TIMER_END 0xffec6ff

/*
 * Macros for device 'hps_0_arm_gic_0', class 'arm_gic'
 * The macros are prefixed with 'HPS_0_ARM_GIC_0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_ARM_GIC_0_COMPONENT_TYPE arm_gic
#define HPS_0_ARM_GIC_0_COMPONENT_NAME hps_0_arm_gic_0
#define HPS_0_ARM_GIC_0_BASE 0xffed000
#define HPS_0_ARM_GIC_0_SPAN 4096
#define HPS_0_ARM_GIC_0_END 0xffedfff

/*
 * Macros for device 'hps_0_L2', class 'L2'
 * The macros are prefixed with 'HPS_0_L2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_L2_COMPONENT_TYPE L2
#define HPS_0_L2_COMPONENT_NAME hps_0_L2
#define HPS_0_L2_BASE 0xffef000
#define HPS_0_L2_SPAN 4096
#define HPS_0_L2_END 0xffeffff

/*
 * Macros for device 'hps_0_axi_ocram', class 'axi_ocram'
 * The macros are prefixed with 'HPS_0_AXI_OCRAM_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_AXI_OCRAM_COMPONENT_TYPE axi_ocram
#define HPS_0_AXI_OCRAM_COMPONENT_NAME hps_0_axi_ocram
#define HPS_0_AXI_OCRAM_BASE 0xffff0000
#define HPS_0_AXI_OCRAM_SPAN 65536
#define HPS_0_AXI_OCRAM_END 0xfffffff
#define HPS_0_AXI_OCRAM_SIZE_MULTIPLE 1
#define HPS_0_AXI_OCRAM_SIZE_VALUE 1<<16
#define HPS_0_AXI_OCRAM_WRITABLE 1

```

```
#define HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_HEX 0
#define HPS_0_AXI_OCRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 16
```

```
#endif /* _ALTERA_HPS_0_ARM_A9_0_H_ */
```

hps_0_arm_a9_1.h

```
#ifndef _ALTERA_HPS_0_ARM_A9_1_H_
#define _ALTERA_HPS_0_ARM_A9_1_H_
```

```
/*
```

```
 * This file was automatically generated by the swinfo2header utility.
```

```
 *
```

```
 * Created from SOPC Builder system 'soc_system' in
```

```
 * file './soc_system.sopcinfo'.
```

```
 */
```

```
/*
```

```
 * This file contains macros for module 'hps_0_arm_a9_1' and devices
```

```
 * connected to the following master:
```

```
 * altera_axi_master
```

```
 *
```

```
 * Do not include this header file and another header file created for a
```

```
 * different module or master group at the same time.
```

```
 * Doing so may result in duplicate macro names.
```

```
 * Instead, use the system header file which has macros with unique names.
```

```
 */
```

```
/*
```

```
 * Macros for device 'hps_0_axi_sdram', class 'axi_sdram'
```

```
 * The macros are prefixed with 'HPS_0_AXI_SDRAM_'.
```

```
 * The prefix is the slave descriptor.
```

```
 */
```

```
#define HPS_0_AXI_SDRAM_COMPONENT_TYPE axi_sdram
```

```
#define HPS_0_AXI_SDRAM_COMPONENT_NAME hps_0_axi_sdram
```

```
#define HPS_0_AXI_SDRAM_BASE 0x0
```

```
#define HPS_0_AXI_SDRAM_SPAN 0x80000000
```

```
#define HPS_0_AXI_SDRAM_END 0x7fffffff
```

```
#define HPS_0_AXI_SDRAM_SIZE_MULTIPLE 1
```

```

#define HPS_0_AXI_SDRAM_SIZE_VALUE 1<<31
#define HPS_0_AXI_SDRAM_WRITABLE 1
#define HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_HEX 0
#define HPS_0_AXI_SDRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 31

/*
 * Macros for device 'div_data_0', class 'div_data'
 * The macros are prefixed with 'DIV_DATA_0_'.
 * The prefix is the slave descriptor.
 */
#define DIV_DATA_0_COMPONENT_TYPE div_data
#define DIV_DATA_0_COMPONENT_NAME div_data_0
#define DIV_DATA_0_BASE 0xff200000
#define DIV_DATA_0_SPAN 64
#define DIV_DATA_0_END 0xff20003f

/*
 * Macros for device 'div_control_0', class 'div_control'
 * The macros are prefixed with 'DIV_CONTROL_0_'.
 * The prefix is the slave descriptor.
 */
#define DIV_CONTROL_0_COMPONENT_TYPE div_control
#define DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define DIV_CONTROL_0_BASE 0xff200040
#define DIV_CONTROL_0_SPAN 64
#define DIV_CONTROL_0_END 0xff20007f

/*
 * Macros for device 'hps_0_gmac0', class 'stmmac'
 * The macros are prefixed with 'HPS_0_GMAC0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GMAC0_COMPONENT_TYPE stmmac
#define HPS_0_GMAC0_COMPONENT_NAME hps_0_gmac0
#define HPS_0_GMAC0_BASE 0xff700000
#define HPS_0_GMAC0_SPAN 8192
#define HPS_0_GMAC0_END 0xff701fff

/*

```

```

* Macros for device 'hps_0_gmac1', class 'stmmac'
* The macros are prefixed with 'HPS_0_GMAC1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_GMAC1_COMPONENT_TYPE stmmac
#define HPS_0_GMAC1_COMPONENT_NAME hps_0_gmac1
#define HPS_0_GMAC1_BASE 0xff702000
#define HPS_0_GMAC1_SPAN 8192
#define HPS_0_GMAC1_END 0xff703fff

/*
* Macros for device 'hps_0_sdmmc', class 'sdmmc'
* The macros are prefixed with 'HPS_0_SDMMC_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_SDMMC_COMPONENT_TYPE sdmmc
#define HPS_0_SDMMC_COMPONENT_NAME hps_0_sdmmc
#define HPS_0_SDMMC_BASE 0xff704000
#define HPS_0_SDMMC_SPAN 4096
#define HPS_0_SDMMC_END 0xff704fff

/*
* Macros for device 'hps_0_qspi', class 'qspi'
* The macros are prefixed with 'HPS_0_QSPI_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_QSPI_COMPONENT_TYPE qspi
#define HPS_0_QSPI_COMPONENT_NAME hps_0_qspi
#define HPS_0_QSPI_BASE 0xff705000
#define HPS_0_QSPI_SPAN 256
#define HPS_0_QSPI_END 0xff7050ff

/*
* Macros for device 'hps_0_fpgamgr', class 'fpgamgr'
* The macros are prefixed with 'HPS_0_FPGAMGR_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_FPGAMGR_COMPONENT_TYPE fpgamgr
#define HPS_0_FPGAMGR_COMPONENT_NAME hps_0_fpgamgr
#define HPS_0_FPGAMGR_BASE 0xff706000

```

```

#define HPS_0_FPGAMGR_SPAN 4096
#define HPS_0_FPGAMGR_END 0xff706fff

/*
 * Macros for device 'hps_0_gpio0', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO0_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO0_COMPONENT_NAME hps_0_gpio0
#define HPS_0_GPIO0_BASE 0xff708000
#define HPS_0_GPIO0_SPAN 256
#define HPS_0_GPIO0_END 0xff7080ff

/*
 * Macros for device 'hps_0_gpio1', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO1_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO1_COMPONENT_NAME hps_0_gpio1
#define HPS_0_GPIO1_BASE 0xff709000
#define HPS_0_GPIO1_SPAN 256
#define HPS_0_GPIO1_END 0xff7090ff

/*
 * Macros for device 'hps_0_gpio2', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO2_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO2_COMPONENT_NAME hps_0_gpio2
#define HPS_0_GPIO2_BASE 0xff70a000
#define HPS_0_GPIO2_SPAN 256
#define HPS_0_GPIO2_END 0xff70a0ff

/*
 * Macros for device 'hps_0_l3regs', class 'l3regs'
 * The macros are prefixed with 'HPS_0_L3REGS_'.
 * The prefix is the slave descriptor.

```

```

*/
#define HPS_0_L3REGS_COMPONENT_TYPE l3regs
#define HPS_0_L3REGS_COMPONENT_NAME hps_0_l3regs
#define HPS_0_L3REGS_BASE 0xff800000
#define HPS_0_L3REGS_SPAN 4096
#define HPS_0_L3REGS_END 0xff800fff

/*
 * Macros for device 'hps_0_nand0', class 'denali_nand'
 * The macros are prefixed with 'HPS_0_NAND0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_NAND0_COMPONENT_TYPE denali_nand
#define HPS_0_NAND0_COMPONENT_NAME hps_0_nand0
#define HPS_0_NAND0_BASE 0xff900000
#define HPS_0_NAND0_SPAN 65536
#define HPS_0_NAND0_END 0xff90ffff

/*
 * Macros for device 'hps_0_usb0', class 'usb'
 * The macros are prefixed with 'HPS_0_USB0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_USB0_COMPONENT_TYPE usb
#define HPS_0_USB0_COMPONENT_NAME hps_0_usb0
#define HPS_0_USB0_BASE 0xffb00000
#define HPS_0_USB0_SPAN 262144
#define HPS_0_USB0_END 0xffb3ffff

/*
 * Macros for device 'hps_0_usb1', class 'usb'
 * The macros are prefixed with 'HPS_0_USB1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_USB1_COMPONENT_TYPE usb
#define HPS_0_USB1_COMPONENT_NAME hps_0_usb1
#define HPS_0_USB1_BASE 0xffb40000
#define HPS_0_USB1_SPAN 262144
#define HPS_0_USB1_END 0xffb7ffff

```

```

/*
 * Macros for device 'hps_0_dcan0', class 'bosch_dcan'
 * The macros are prefixed with 'HPS_0_DCAN0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_DCAN0_COMPONENT_TYPE bosch_dcan
#define HPS_0_DCAN0_COMPONENT_NAME hps_0_dcan0
#define HPS_0_DCAN0_BASE 0xffc00000
#define HPS_0_DCAN0_SPAN 4096
#define HPS_0_DCAN0_END 0xffc00fff

/*
 * Macros for device 'hps_0_dcan1', class 'bosch_dcan'
 * The macros are prefixed with 'HPS_0_DCAN1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_DCAN1_COMPONENT_TYPE bosch_dcan
#define HPS_0_DCAN1_COMPONENT_NAME hps_0_dcan1
#define HPS_0_DCAN1_BASE 0xffc01000
#define HPS_0_DCAN1_SPAN 4096
#define HPS_0_DCAN1_END 0xffc01fff

/*
 * Macros for device 'hps_0_uart0', class 'snps_uart'
 * The macros are prefixed with 'HPS_0_UART0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_UART0_COMPONENT_TYPE snps_uart
#define HPS_0_UART0_COMPONENT_NAME hps_0_uart0
#define HPS_0_UART0_BASE 0xffc02000
#define HPS_0_UART0_SPAN 256
#define HPS_0_UART0_END 0xffc020ff
#define HPS_0_UART0_FIFO_DEPTH 128
#define HPS_0_UART0_FIFO_HWFC 0
#define HPS_0_UART0_FIFO_MODE 1
#define HPS_0_UART0_FIFO_SWFC 0
#define HPS_0_UART0_FREQ 100000000

/*
 * Macros for device 'hps_0_uart1', class 'snps_uart'

```

```

* The macros are prefixed with 'HPS_0_UART1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_UART1_COMPONENT_TYPE snps_uart
#define HPS_0_UART1_COMPONENT_NAME hps_0_uart1
#define HPS_0_UART1_BASE 0xffc03000
#define HPS_0_UART1_SPAN 256
#define HPS_0_UART1_END 0xffc030ff
#define HPS_0_UART1_FIFO_DEPTH 128
#define HPS_0_UART1_FIFO_HWFC 0
#define HPS_0_UART1_FIFO_MODE 1
#define HPS_0_UART1_FIFO_SWFC 0
#define HPS_0_UART1_FREQ 100000000

/*
* Macros for device 'hps_0_i2c0', class 'designware_i2c'
* The macros are prefixed with 'HPS_0_I2C0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_I2C0_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C0_COMPONENT_NAME hps_0_i2c0
#define HPS_0_I2C0_BASE 0xffc04000
#define HPS_0_I2C0_SPAN 256
#define HPS_0_I2C0_END 0xffc040ff

/*
* Macros for device 'hps_0_i2c1', class 'designware_i2c'
* The macros are prefixed with 'HPS_0_I2C1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_I2C1_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C1_COMPONENT_NAME hps_0_i2c1
#define HPS_0_I2C1_BASE 0xffc05000
#define HPS_0_I2C1_SPAN 256
#define HPS_0_I2C1_END 0xffc050ff

/*
* Macros for device 'hps_0_i2c2', class 'designware_i2c'
* The macros are prefixed with 'HPS_0_I2C2_'.
* The prefix is the slave descriptor.

```

```

*/
#define HPS_0_I2C2_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C2_COMPONENT_NAME hps_0_i2c2
#define HPS_0_I2C2_BASE 0xffc06000
#define HPS_0_I2C2_SPAN 256
#define HPS_0_I2C2_END 0xffc060ff

/*
* Macros for device 'hps_0_i2c3', class 'designware_i2c'
* The macros are prefixed with 'HPS_0_I2C3_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_I2C3_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C3_COMPONENT_NAME hps_0_i2c3
#define HPS_0_I2C3_BASE 0xffc07000
#define HPS_0_I2C3_SPAN 256
#define HPS_0_I2C3_END 0xffc070ff

/*
* Macros for device 'hps_0_timer0', class 'dw_apb_timer_sp'
* The macros are prefixed with 'HPS_0_TIMER0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_TIMER0_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_TIMER0_COMPONENT_NAME hps_0_timer0
#define HPS_0_TIMER0_BASE 0xffc08000
#define HPS_0_TIMER0_SPAN 256
#define HPS_0_TIMER0_END 0xffc080ff

/*
* Macros for device 'hps_0_timer1', class 'dw_apb_timer_sp'
* The macros are prefixed with 'HPS_0_TIMER1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_TIMER1_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_TIMER1_COMPONENT_NAME hps_0_timer1
#define HPS_0_TIMER1_BASE 0xffc09000
#define HPS_0_TIMER1_SPAN 256
#define HPS_0_TIMER1_END 0xffc090ff

```

```

/*
 * Macros for device 'hps_0_sdrctl', class 'sdrctl'
 * The macros are prefixed with 'HPS_0_SDRCTL_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SDRCTL_COMPONENT_TYPE sdrctl
#define HPS_0_SDRCTL_COMPONENT_NAME hps_0_sdrctl
#define HPS_0_SDRCTL_BASE 0xffc25000
#define HPS_0_SDRCTL_SPAN 4096
#define HPS_0_SDRCTL_END 0xffc25fff

/*
 * Macros for device 'hps_0_timer2', class 'dw_apb_timer_osc'
 * The macros are prefixed with 'HPS_0_TIMER2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER2_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_TIMER2_COMPONENT_NAME hps_0_timer2
#define HPS_0_TIMER2_BASE 0xffd00000
#define HPS_0_TIMER2_SPAN 256
#define HPS_0_TIMER2_END 0xffd000ff

/*
 * Macros for device 'hps_0_timer3', class 'dw_apb_timer_osc'
 * The macros are prefixed with 'HPS_0_TIMER3_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER3_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_TIMER3_COMPONENT_NAME hps_0_timer3
#define HPS_0_TIMER3_BASE 0xffd01000
#define HPS_0_TIMER3_SPAN 256
#define HPS_0_TIMER3_END 0xffd010ff

/*
 * Macros for device 'hps_0_wd_timer0', class 'dw_wd_timer'
 * The macros are prefixed with 'HPS_0_WD_TIMER0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_WD_TIMER0_COMPONENT_TYPE dw_wd_timer
#define HPS_0_WD_TIMER0_COMPONENT_NAME hps_0_wd_timer0

```

```

#define HPS_0_WD_TIMER0_BASE 0xffd02000
#define HPS_0_WD_TIMER0_SPAN 256
#define HPS_0_WD_TIMER0_END 0xffd020ff

/*
 * Macros for device 'hps_0_wd_timer1', class 'dw_wd_timer'
 * The macros are prefixed with 'HPS_0_WD_TIMER1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_WD_TIMER1_COMPONENT_TYPE dw_wd_timer
#define HPS_0_WD_TIMER1_COMPONENT_NAME hps_0_wd_timer1
#define HPS_0_WD_TIMER1_BASE 0xffd03000
#define HPS_0_WD_TIMER1_SPAN 256
#define HPS_0_WD_TIMER1_END 0xffd030ff

/*
 * Macros for device 'hps_0_clkmgr', class 'clkmgr'
 * The macros are prefixed with 'HPS_0_CLKMGR_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_CLKMGR_COMPONENT_TYPE clkmgr
#define HPS_0_CLKMGR_COMPONENT_NAME hps_0_clkmgr
#define HPS_0_CLKMGR_BASE 0xffd04000
#define HPS_0_CLKMGR_SPAN 4096
#define HPS_0_CLKMGR_END 0xffd04fff

/*
 * Macros for device 'hps_0_rstmgr', class 'rstmgr'
 * The macros are prefixed with 'HPS_0_RSTMGR_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_RSTMGR_COMPONENT_TYPE rstmgr
#define HPS_0_RSTMGR_COMPONENT_NAME hps_0_rstmgr
#define HPS_0_RSTMGR_BASE 0xffd05000
#define HPS_0_RSTMGR_SPAN 256
#define HPS_0_RSTMGR_END 0xffd050ff

/*
 * Macros for device 'hps_0_sysmgr', class 'sysmgr'
 * The macros are prefixed with 'HPS_0_SYSMGR_'.

```

```

* The prefix is the slave descriptor.
*/
#define HPS_0_SYSMGR_COMPONENT_TYPE sysmgr
#define HPS_0_SYSMGR_COMPONENT_NAME hps_0_sysmgr
#define HPS_0_SYSMGR_BASE 0xffd08000
#define HPS_0_SYSMGR_SPAN 1024
#define HPS_0_SYSMGR_END 0xffd083ff

/*
* Macros for device 'hps_0_dma', class 'dma'
* The macros are prefixed with 'HPS_0_DMA_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_DMA_COMPONENT_TYPE dma
#define HPS_0_DMA_COMPONENT_NAME hps_0_dma
#define HPS_0_DMA_BASE 0xffe01000
#define HPS_0_DMA_SPAN 4096
#define HPS_0_DMA_END 0xffe01fff

/*
* Macros for device 'hps_0_spim0', class 'spi'
* The macros are prefixed with 'HPS_0_SPIM0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_SPIM0_COMPONENT_TYPE spi
#define HPS_0_SPIM0_COMPONENT_NAME hps_0_spim0
#define HPS_0_SPIM0_BASE 0xfff00000
#define HPS_0_SPIM0_SPAN 256
#define HPS_0_SPIM0_END 0xfff000ff

/*
* Macros for device 'hps_0_spim1', class 'spi'
* The macros are prefixed with 'HPS_0_SPIM1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_SPIM1_COMPONENT_TYPE spi
#define HPS_0_SPIM1_COMPONENT_NAME hps_0_spim1
#define HPS_0_SPIM1_BASE 0xfff01000
#define HPS_0_SPIM1_SPAN 256
#define HPS_0_SPIM1_END 0xfff010ff

```

```

/*
 * Macros for device 'hps_0_scu', class 'scu'
 * The macros are prefixed with 'HPS_0_SCU_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SCU_COMPONENT_TYPE scu
#define HPS_0_SCU_COMPONENT_NAME hps_0_scu
#define HPS_0_SCU_BASE 0xffec000
#define HPS_0_SCU_SPAN 256
#define HPS_0_SCU_END 0xffec0ff

/*
 * Macros for device 'hps_0_timer', class 'timer'
 * The macros are prefixed with 'HPS_0_TIMER_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER_COMPONENT_TYPE timer
#define HPS_0_TIMER_COMPONENT_NAME hps_0_timer
#define HPS_0_TIMER_BASE 0xffec600
#define HPS_0_TIMER_SPAN 256
#define HPS_0_TIMER_END 0xffec6ff

/*
 * Macros for device 'hps_0_arm_gic_0', class 'arm_gic'
 * The macros are prefixed with 'HPS_0_ARM_GIC_0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_ARM_GIC_0_COMPONENT_TYPE arm_gic
#define HPS_0_ARM_GIC_0_COMPONENT_NAME hps_0_arm_gic_0
#define HPS_0_ARM_GIC_0_BASE 0xffed000
#define HPS_0_ARM_GIC_0_SPAN 4096
#define HPS_0_ARM_GIC_0_END 0xffedfff

/*
 * Macros for device 'hps_0_L2', class 'L2'
 * The macros are prefixed with 'HPS_0_L2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_L2_COMPONENT_TYPE L2

```

```

#define HPS_0_L2_COMPONENT_NAME hps_0_L2
#define HPS_0_L2_BASE 0xffef000
#define HPS_0_L2_SPAN 4096
#define HPS_0_L2_END 0xffefffff

/*
 * Macros for device 'hps_0_axi_ocram', class 'axi_ocram'
 * The macros are prefixed with 'HPS_0_AXI_OCRAM_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_AXI_OCRAM_COMPONENT_TYPE axi_ocram
#define HPS_0_AXI_OCRAM_COMPONENT_NAME hps_0_axi_ocram
#define HPS_0_AXI_OCRAM_BASE 0xffff0000
#define HPS_0_AXI_OCRAM_SPAN 65536
#define HPS_0_AXI_OCRAM_END 0xffffffff
#define HPS_0_AXI_OCRAM_SIZE_MULTIPLE 1
#define HPS_0_AXI_OCRAM_SIZE_VALUE 1<<16
#define HPS_0_AXI_OCRAM_WRITABLE 1
#define HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_HEX 0
#define HPS_0_AXI_OCRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 16

#endif /* _ALTERA_HPS_0_ARM_A9_1_H_ */

```

hps_0_bridges.h

```

#ifndef _ALTERA_HPS_0_BRIDGES_H_
#define _ALTERA_HPS_0_BRIDGES_H_

/*
 * This file was automatically generated by the swinfo2header utility.
 *
 * Created from SOPC Builder system 'soc_system' in
 * file './soc_system.sopcinfo'.
 */

/*
 * This file contains macros for module 'hps_0_bridges' and devices
 * connected to the following masters:
 * axi_f2h

```

```

* h2f
* h2f_lw
*
* Do not include this header file and another header file created for a
* different module or master group at the same time.
* Doing so may result in duplicate macro names.
* Instead, use the system header file which has macros with unique names.
*/

/*
* Macros for device 'hps_0_axi_sdrām', class 'axi_sdrām'
* The macros are prefixed with 'HPS_0_AXI_SDRAM_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_AXI_SDRAM_COMPONENT_TYPE axi_sdrām
#define HPS_0_AXI_SDRAM_COMPONENT_NAME hps_0_axi_sdrām
#define HPS_0_AXI_SDRAM_BASE 0x0
#define HPS_0_AXI_SDRAM_SPAN 0x80000000
#define HPS_0_AXI_SDRAM_END 0x7ffffff
#define HPS_0_AXI_SDRAM_SIZE_MULTIPLE 1
#define HPS_0_AXI_SDRAM_SIZE_VALUE 1<<31
#define HPS_0_AXI_SDRAM_WRITABLE 1
#define HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_AXI_SDRAM_MEMORY_INFO_GENERATE_HEX 0
#define HPS_0_AXI_SDRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 31

/*
* Macros for device 'div_data_0', class 'div_data'
* The macros are prefixed with 'DIV_DATA_0_'.
* The prefix is the slave descriptor.
*/
#define DIV_DATA_0_COMPONENT_TYPE div_data
#define DIV_DATA_0_COMPONENT_NAME div_data_0
#define DIV_DATA_0_BASE 0x0
#define DIV_DATA_0_SPAN 64
#define DIV_DATA_0_END 0x3f

/*
* Macros for device 'div_control_0', class 'div_control'
* The macros are prefixed with 'DIV_CONTROL_0_'.

```

```

* The prefix is the slave descriptor.
*/
#define DIV_CONTROL_0_COMPONENT_TYPE div_control
#define DIV_CONTROL_0_COMPONENT_NAME div_control_0
#define DIV_CONTROL_0_BASE 0x40
#define DIV_CONTROL_0_SPAN 64
#define DIV_CONTROL_0_END 0x7f

/*
* Macros for device 'hps_0_gmac0', class 'stmmac'
* The macros are prefixed with 'HPS_0_GMAC0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_GMAC0_COMPONENT_TYPE stmmac
#define HPS_0_GMAC0_COMPONENT_NAME hps_0_gmac0
#define HPS_0_GMAC0_BASE 0xff700000
#define HPS_0_GMAC0_SPAN 8192
#define HPS_0_GMAC0_END 0xff701fff

/*
* Macros for device 'hps_0_gmac1', class 'stmmac'
* The macros are prefixed with 'HPS_0_GMAC1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_GMAC1_COMPONENT_TYPE stmmac
#define HPS_0_GMAC1_COMPONENT_NAME hps_0_gmac1
#define HPS_0_GMAC1_BASE 0xff702000
#define HPS_0_GMAC1_SPAN 8192
#define HPS_0_GMAC1_END 0xff703fff

/*
* Macros for device 'hps_0_sdmmc', class 'sdmmc'
* The macros are prefixed with 'HPS_0_SDMMC_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_SDMMC_COMPONENT_TYPE sdmmc
#define HPS_0_SDMMC_COMPONENT_NAME hps_0_sdmmc
#define HPS_0_SDMMC_BASE 0xff704000
#define HPS_0_SDMMC_SPAN 4096
#define HPS_0_SDMMC_END 0xff704fff

```

```

/*
 * Macros for device 'hps_0_qspi', class 'qspi'
 * The macros are prefixed with 'HPS_0_QSPI_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_QSPI_COMPONENT_TYPE qspi
#define HPS_0_QSPI_COMPONENT_NAME hps_0_qspi
#define HPS_0_QSPI_BASE 0xff705000
#define HPS_0_QSPI_SPAN 256
#define HPS_0_QSPI_END 0xff7050ff

/*
 * Macros for device 'hps_0_fpgamgr', class 'fpgamgr'
 * The macros are prefixed with 'HPS_0_FPGAMGR_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_FPGAMGR_COMPONENT_TYPE fpgamgr
#define HPS_0_FPGAMGR_COMPONENT_NAME hps_0_fpgamgr
#define HPS_0_FPGAMGR_BASE 0xff706000
#define HPS_0_FPGAMGR_SPAN 4096
#define HPS_0_FPGAMGR_END 0xff706fff

/*
 * Macros for device 'hps_0_gpio0', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO0_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO0_COMPONENT_NAME hps_0_gpio0
#define HPS_0_GPIO0_BASE 0xff708000
#define HPS_0_GPIO0_SPAN 256
#define HPS_0_GPIO0_END 0xff7080ff

/*
 * Macros for device 'hps_0_gpio1', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO1_COMPONENT_TYPE dw_gpio

```

```

#define HPS_0_GPIO1_COMPONENT_NAME hps_0_gpio1
#define HPS_0_GPIO1_BASE 0xff709000
#define HPS_0_GPIO1_SPAN 256
#define HPS_0_GPIO1_END 0xff7090ff

/*
 * Macros for device 'hps_0_gpio2', class 'dw_gpio'
 * The macros are prefixed with 'HPS_0_GPIO2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_GPIO2_COMPONENT_TYPE dw_gpio
#define HPS_0_GPIO2_COMPONENT_NAME hps_0_gpio2
#define HPS_0_GPIO2_BASE 0xff70a000
#define HPS_0_GPIO2_SPAN 256
#define HPS_0_GPIO2_END 0xff70a0ff

/*
 * Macros for device 'hps_0_l3regs', class 'l3regs'
 * The macros are prefixed with 'HPS_0_L3REGS_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_L3REGS_COMPONENT_TYPE l3regs
#define HPS_0_L3REGS_COMPONENT_NAME hps_0_l3regs
#define HPS_0_L3REGS_BASE 0xff800000
#define HPS_0_L3REGS_SPAN 4096
#define HPS_0_L3REGS_END 0xff800fff

/*
 * Macros for device 'hps_0_nand0', class 'denali_nand'
 * The macros are prefixed with 'HPS_0_NAND0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_NAND0_COMPONENT_TYPE denali_nand
#define HPS_0_NAND0_COMPONENT_NAME hps_0_nand0
#define HPS_0_NAND0_BASE 0xff900000
#define HPS_0_NAND0_SPAN 65536
#define HPS_0_NAND0_END 0xff90ffff

/*
 * Macros for device 'hps_0_usb0', class 'usb'

```

```

* The macros are prefixed with 'HPS_0_USB0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_USB0_COMPONENT_TYPE usb
#define HPS_0_USB0_COMPONENT_NAME hps_0_usb0
#define HPS_0_USB0_BASE 0xffb00000
#define HPS_0_USB0_SPAN 262144
#define HPS_0_USB0_END 0xffb3fff

/*
* Macros for device 'hps_0_usb1', class 'usb'
* The macros are prefixed with 'HPS_0_USB1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_USB1_COMPONENT_TYPE usb
#define HPS_0_USB1_COMPONENT_NAME hps_0_usb1
#define HPS_0_USB1_BASE 0xffb40000
#define HPS_0_USB1_SPAN 262144
#define HPS_0_USB1_END 0xffb7fff

/*
* Macros for device 'hps_0_dcan0', class 'bosch_dcan'
* The macros are prefixed with 'HPS_0_DCAN0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_DCAN0_COMPONENT_TYPE bosch_dcan
#define HPS_0_DCAN0_COMPONENT_NAME hps_0_dcan0
#define HPS_0_DCAN0_BASE 0xffc00000
#define HPS_0_DCAN0_SPAN 4096
#define HPS_0_DCAN0_END 0xffc00fff

/*
* Macros for device 'hps_0_dcan1', class 'bosch_dcan'
* The macros are prefixed with 'HPS_0_DCAN1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_DCAN1_COMPONENT_TYPE bosch_dcan
#define HPS_0_DCAN1_COMPONENT_NAME hps_0_dcan1
#define HPS_0_DCAN1_BASE 0xffc01000
#define HPS_0_DCAN1_SPAN 4096

```

```

#define HPS_0_DCAN1_END 0xffc01fff

/*
 * Macros for device 'hps_0_uart0', class 'snps_uart'
 * The macros are prefixed with 'HPS_0_UART0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_UART0_COMPONENT_TYPE snps_uart
#define HPS_0_UART0_COMPONENT_NAME hps_0_uart0
#define HPS_0_UART0_BASE 0xffc02000
#define HPS_0_UART0_SPAN 256
#define HPS_0_UART0_END 0xffc020ff
#define HPS_0_UART0_FIFO_DEPTH 128
#define HPS_0_UART0_FIFO_HWFC 0
#define HPS_0_UART0_FIFO_MODE 1
#define HPS_0_UART0_FIFO_SWFC 0
#define HPS_0_UART0_FREQ 100000000

/*
 * Macros for device 'hps_0_uart1', class 'snps_uart'
 * The macros are prefixed with 'HPS_0_UART1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_UART1_COMPONENT_TYPE snps_uart
#define HPS_0_UART1_COMPONENT_NAME hps_0_uart1
#define HPS_0_UART1_BASE 0xffc03000
#define HPS_0_UART1_SPAN 256
#define HPS_0_UART1_END 0xffc030ff
#define HPS_0_UART1_FIFO_DEPTH 128
#define HPS_0_UART1_FIFO_HWFC 0
#define HPS_0_UART1_FIFO_MODE 1
#define HPS_0_UART1_FIFO_SWFC 0
#define HPS_0_UART1_FREQ 100000000

/*
 * Macros for device 'hps_0_i2c0', class 'designware_i2c'
 * The macros are prefixed with 'HPS_0_I2C0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_I2C0_COMPONENT_TYPE designware_i2c

```

```

#define HPS_0_I2C0_COMPONENT_NAME hps_0_i2c0
#define HPS_0_I2C0_BASE 0xffc04000
#define HPS_0_I2C0_SPAN 256
#define HPS_0_I2C0_END 0xffc040ff

/*
 * Macros for device 'hps_0_i2c1', class 'designware_i2c'
 * The macros are prefixed with 'HPS_0_I2C1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_I2C1_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C1_COMPONENT_NAME hps_0_i2c1
#define HPS_0_I2C1_BASE 0xffc05000
#define HPS_0_I2C1_SPAN 256
#define HPS_0_I2C1_END 0xffc050ff

/*
 * Macros for device 'hps_0_i2c2', class 'designware_i2c'
 * The macros are prefixed with 'HPS_0_I2C2_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_I2C2_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C2_COMPONENT_NAME hps_0_i2c2
#define HPS_0_I2C2_BASE 0xffc06000
#define HPS_0_I2C2_SPAN 256
#define HPS_0_I2C2_END 0xffc060ff

/*
 * Macros for device 'hps_0_i2c3', class 'designware_i2c'
 * The macros are prefixed with 'HPS_0_I2C3_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_I2C3_COMPONENT_TYPE designware_i2c
#define HPS_0_I2C3_COMPONENT_NAME hps_0_i2c3
#define HPS_0_I2C3_BASE 0xffc07000
#define HPS_0_I2C3_SPAN 256
#define HPS_0_I2C3_END 0xffc070ff

/*
 * Macros for device 'hps_0_timer0', class 'dw_apb_timer_sp'

```

```

* The macros are prefixed with 'HPS_0_TIMER0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_TIMER0_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_TIMER0_COMPONENT_NAME hps_0_timer0
#define HPS_0_TIMER0_BASE 0xffc08000
#define HPS_0_TIMER0_SPAN 256
#define HPS_0_TIMER0_END 0xffc080ff

/*
* Macros for device 'hps_0_timer1', class 'dw_apb_timer_sp'
* The macros are prefixed with 'HPS_0_TIMER1_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_TIMER1_COMPONENT_TYPE dw_apb_timer_sp
#define HPS_0_TIMER1_COMPONENT_NAME hps_0_timer1
#define HPS_0_TIMER1_BASE 0xffc09000
#define HPS_0_TIMER1_SPAN 256
#define HPS_0_TIMER1_END 0xffc090ff

/*
* Macros for device 'hps_0_sdrctl', class 'sdrctl'
* The macros are prefixed with 'HPS_0_SDRCTL_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_SDRCTL_COMPONENT_TYPE sdrctl
#define HPS_0_SDRCTL_COMPONENT_NAME hps_0_sdrctl
#define HPS_0_SDRCTL_BASE 0xffc25000
#define HPS_0_SDRCTL_SPAN 4096
#define HPS_0_SDRCTL_END 0xffc25fff

/*
* Macros for device 'hps_0_timer2', class 'dw_apb_timer_osc'
* The macros are prefixed with 'HPS_0_TIMER2_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_TIMER2_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_TIMER2_COMPONENT_NAME hps_0_timer2
#define HPS_0_TIMER2_BASE 0xffd00000
#define HPS_0_TIMER2_SPAN 256

```

```

#define HPS_0_TIMER2_END 0xffd000ff

/*
 * Macros for device 'hps_0_timer3', class 'dw_apb_timer_osc'
 * The macros are prefixed with 'HPS_0_TIMER3_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_TIMER3_COMPONENT_TYPE dw_apb_timer_osc
#define HPS_0_TIMER3_COMPONENT_NAME hps_0_timer3
#define HPS_0_TIMER3_BASE 0xffd01000
#define HPS_0_TIMER3_SPAN 256
#define HPS_0_TIMER3_END 0xffd010ff

/*
 * Macros for device 'hps_0_clkmgr', class 'clkmgr'
 * The macros are prefixed with 'HPS_0_CLKMGR_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_CLKMGR_COMPONENT_TYPE clkmgr
#define HPS_0_CLKMGR_COMPONENT_NAME hps_0_clkmgr
#define HPS_0_CLKMGR_BASE 0xffd04000
#define HPS_0_CLKMGR_SPAN 4096
#define HPS_0_CLKMGR_END 0xffd04fff

/*
 * Macros for device 'hps_0_rstmgr', class 'rstmgr'
 * The macros are prefixed with 'HPS_0_RSTMGR_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_RSTMGR_COMPONENT_TYPE rstmgr
#define HPS_0_RSTMGR_COMPONENT_NAME hps_0_rstmgr
#define HPS_0_RSTMGR_BASE 0xffd05000
#define HPS_0_RSTMGR_SPAN 256
#define HPS_0_RSTMGR_END 0xffd050ff

/*
 * Macros for device 'hps_0_sysmgr', class 'sysmgr'
 * The macros are prefixed with 'HPS_0_SYSMGR_'.
 * The prefix is the slave descriptor.
 */

```

```

#define HPS_0_SYSMGR_COMPONENT_TYPE sysmgr
#define HPS_0_SYSMGR_COMPONENT_NAME hps_0_sysmgr
#define HPS_0_SYSMGR_BASE 0xffd08000
#define HPS_0_SYSMGR_SPAN 1024
#define HPS_0_SYSMGR_END 0xffd083ff

/*
 * Macros for device 'hps_0_dma', class 'dma'
 * The macros are prefixed with 'HPS_0_DMA_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_DMA_COMPONENT_TYPE dma
#define HPS_0_DMA_COMPONENT_NAME hps_0_dma
#define HPS_0_DMA_BASE 0xffe01000
#define HPS_0_DMA_SPAN 4096
#define HPS_0_DMA_END 0xffe01fff

/*
 * Macros for device 'hps_0_spim0', class 'spi'
 * The macros are prefixed with 'HPS_0_SPIM0_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SPIM0_COMPONENT_TYPE spi
#define HPS_0_SPIM0_COMPONENT_NAME hps_0_spim0
#define HPS_0_SPIM0_BASE 0xfff00000
#define HPS_0_SPIM0_SPAN 256
#define HPS_0_SPIM0_END 0xfff000ff

/*
 * Macros for device 'hps_0_spim1', class 'spi'
 * The macros are prefixed with 'HPS_0_SPIM1_'.
 * The prefix is the slave descriptor.
 */
#define HPS_0_SPIM1_COMPONENT_TYPE spi
#define HPS_0_SPIM1_COMPONENT_NAME hps_0_spim1
#define HPS_0_SPIM1_BASE 0xfff01000
#define HPS_0_SPIM1_SPAN 256
#define HPS_0_SPIM1_END 0xfff010ff

/*

```

```

* Macros for device 'hps_0_timer', class 'timer'
* The macros are prefixed with 'HPS_0_TIMER_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_TIMER_COMPONENT_TYPE timer
#define HPS_0_TIMER_COMPONENT_NAME hps_0_timer
#define HPS_0_TIMER_BASE 0xfffec600
#define HPS_0_TIMER_SPAN 256
#define HPS_0_TIMER_END 0xfffec6ff

/*
* Macros for device 'hps_0_arm_gic_0', class 'arm_gic'
* The macros are prefixed with 'HPS_0_ARM_GIC_0_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_ARM_GIC_0_COMPONENT_TYPE arm_gic
#define HPS_0_ARM_GIC_0_COMPONENT_NAME hps_0_arm_gic_0
#define HPS_0_ARM_GIC_0_BASE 0xfffed000
#define HPS_0_ARM_GIC_0_SPAN 4096
#define HPS_0_ARM_GIC_0_END 0xfffedfff

/*
* Macros for device 'hps_0_L2', class 'L2'
* The macros are prefixed with 'HPS_0_L2_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_L2_COMPONENT_TYPE L2
#define HPS_0_L2_COMPONENT_NAME hps_0_L2
#define HPS_0_L2_BASE 0xffef000
#define HPS_0_L2_SPAN 4096
#define HPS_0_L2_END 0xffefffff

/*
* Macros for device 'hps_0_axi_ocram', class 'axi_ocram'
* The macros are prefixed with 'HPS_0_AXI_OCRAM_'.
* The prefix is the slave descriptor.
*/
#define HPS_0_AXI_OCRAM_COMPONENT_TYPE axi_ocram
#define HPS_0_AXI_OCRAM_COMPONENT_NAME hps_0_axi_ocram
#define HPS_0_AXI_OCRAM_BASE 0xffff0000

```

```

#define HPS_0_AXI_OCRAM_SPAN 65536
#define HPS_0_AXI_OCRAM_END 0xffffffff
#define HPS_0_AXI_OCRAM_SIZE_MULTIPLE 1
#define HPS_0_AXI_OCRAM_SIZE_VALUE 1<<16
#define HPS_0_AXI_OCRAM_WRITABLE 1
#define HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_DAT_SYM 0
#define HPS_0_AXI_OCRAM_MEMORY_INFO_GENERATE_HEX 0
#define HPS_0_AXI_OCRAM_MEMORY_INFO_MEM_INIT_DATA_WIDTH 16

```

```

#endif /* _ALTERA_HPS_0_BRIDGES_H_ */

```

main.c

```

/* COE838 - System-on-Chip
 * Lab 4 - Custom DIVIDER IP for HPS/FPGA Systems
 * main.c
 */

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>
#include <sys/mman.h>
#include <stdint.h>
#include "hwlib.h"
#include "socal/socal.h"
#include "socal/hps.h"
#include "socal/alt_gpio.h"
#include "hps_0.h"

/* Memory region used for lightweight HPS-to-FPGA bridge */
#define LW_SIZE      0x00200000
#define LWHPS2FPGA_BASE 0xff200000

/* Pointers to divider IP register blocks */
volatile uint32_t *div_control = NULL;
volatile uint32_t *div_data   = NULL;

/* Counters used to track test success */

```

```

int div_success = 0;
int div_total = 0;

/* Reset the divider hardware block */
void reset_system() {
    alt_write_word(div_control + 1, 0x1); // assert reset
    alt_write_word(div_control + 1, 0x0); // release reset
    usleep(1000);
}

/* Send numerator/denominator inputs and trigger division */
void start_div(uint32_t numerator_val, uint32_t denominator_val) {

    alt_write_word(div_data + 0, denominator_val);
    alt_write_word(div_data + 1, numerator_val);

    /* Start pulse for the divider */
    alt_write_word(div_control + 0, 0x1);
    alt_write_word(div_control + 0, 0x0);
}

/* Read results from the divider and verify if its correct */
void read_output(uint32_t num_input, uint32_t den_input) {

    uint32_t quotient_val, remainder_val;
    uint32_t den_echo, num_echo;
    uint32_t expected_q, expected_r;

    /* Wait until hardware sets the DONE signal */
    while (!(alt_read_word(div_control + 2) & 0x1));

    /* Read divider outputs and echoed inputs */
    quotient_val = alt_read_word(div_data + 0);
    remainder_val = alt_read_word(div_data + 1);
    den_echo = alt_read_word(div_data + 2);
    num_echo = alt_read_word(div_data + 3);

```

```

/* Compute expected result using software given specific software safety checks */
if (den_input == 0) {
    expected_q = 0xFFFF;
    expected_r = num_input;
}
else if (num_input == 0) {
    expected_q = 0;
    expected_r = 0;
}
else if (num_input < den_input) {
    expected_q = 0;
    expected_r = num_input;
}
else {
    expected_q = num_input / den_input;
    expected_r = num_input % den_input;
}

printf("DIV Start    : Numerator = 0x%08X (%u), Denominator = 0x%08X (%u)\n",
    num_input, num_input, den_input, den_input);

printf("DIV Result    : Quotient = 0x%08X (%u), Remainder = 0x%08X (%u)\n",
    quotient_val, quotient_val, remainder_val, remainder_val);

printf("Expected Result : Quotient = 0x%08X (%u), Remainder = 0x%08X (%u)\n",
    expected_q, expected_q, expected_r, expected_r);

/* Compare FPGA result with expected software result */
if ((quotient_val == expected_q) && (remainder_val == expected_r)) {
    printf("[SUCCESS]\n");
    div_success++;
} else {
    printf("[FAIL]\n");
}

div_total++;
printf("-----\n");

usleep(1000);
}

```

```

int main(int argc, char **argv) {

    int mem_fd;
    int case_idx;
    void *lw_virtual_base;

    div_success = 0;
    div_total = 0;

    /* Open physical memory to access FPGA registers */
    mem_fd = open("/dev/mem", (O_RDWR | O_SYNC));
    if (mem_fd == -1) {
        printf("ERROR: could not open /dev/mem...\n");
        return 1;
    }

    /* Map FPGA lightweight bridge region into user space */
    lw_virtual_base = mmap(NULL, LW_SIZE,
        (PROT_READ | PROT_WRITE),
        MAP_SHARED,
        mem_fd,
        LWHPS2FPGA_BASE);

    if (lw_virtual_base == MAP_FAILED) {
        printf("ERROR: mmap() failed...\n");
        close(mem_fd);
        return 1;
    }

    /* Assign divider IP register addresses */
    div_control = lw_virtual_base + DIV_CONTROL_0_BASE;
    div_data = lw_virtual_base + DIV_DATA_0_BASE;

    printf("----- FPGA Divider Test ----- \n");
    printf("----->Finished initializing HPS/FPGA system<----- \n");

    /* Run through generated divider cases */
    for (case_idx = 0; case_idx < 30; case_idx++) {

```

```

uint32_t numerator_val = case_idx * 8 + 5;
uint32_t denominator_val = (case_idx % 9) + 1;

reset_system();
start_div(numerator_val, denominator_val);
read_output(numerator_val, denominator_val);
}

/* Final summary of results */
printf("Final Result: %d/%d correct\n", div_success, div_total);

if (div_success == div_total) {
    printf("All tests passed.\n");
} else {
    printf("Some tests failed.\n");
}

/* Cleanup memory mapping */
if (munmap(lw_virtual_base, LW_SIZE) != 0) {
    printf("ERROR: munmap() failed...\n");
    close(mem_fd);
    return 1;
}

close(mem_fd);

return 0;
}

```